

La téléphonie d'entreprise, reconstruite de bout en bout.

Rapport technique — conception, sécurisation et exploitation d'une plateforme VoIP souveraine : PBX Asterisk 20 / FreePBX 17, softphone Flutter multiplateforme, provisionnement par QR, VPN WireGuard et supervision temps réel.

10

EXTENSIONS PJSIP
1001 – 1010

2

PROFILS MÉDIA
SRTP & DTLS-WEbrtc

4

PHASES DE DÉPLOIEMENT
ADRESSAGE → SÉCURITÉ

3

PATTES RÉSEAU
MGMT · VLAN 10 · VPN

8

COUCHES DE SÉCURITÉ
L0 → L7

17

ÉTAPES DE BOOT
AUTOMATISÉES (SYSTEMD)

< 2 min

ONBOARDING UTILISATEUR
E-MAIL → QR → APPEL

0

BACKEND CLOUD
TOUT VIT SUR LE PBX

DÉPÔT SERVEUR

github.com/Asaph-D/serveur — branche main

DÉPÔT CLIENT

github.com/Asaph-D/asaphone — Flutter (Android · Windows · iOS)

CŒUR VOIP

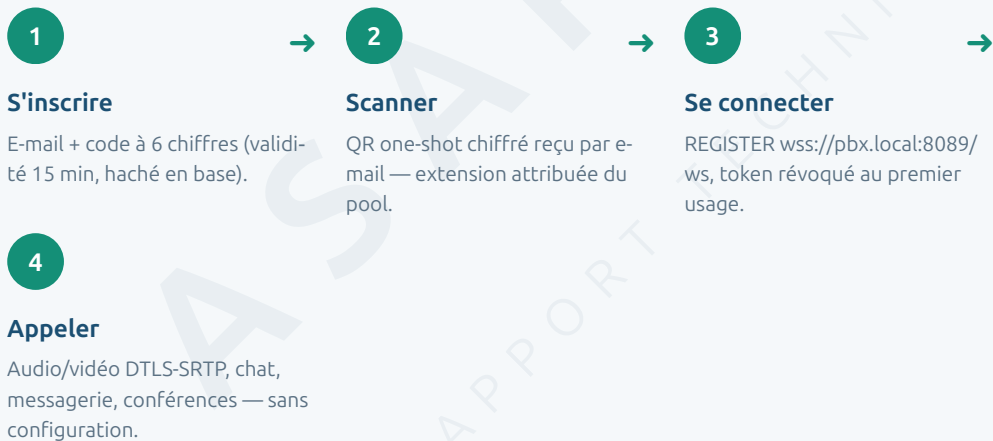
Asterisk 20 LTS (réf. 20.18.2) · FreePBX 17 · PJSIP · ConfBridge · WebRTC

SUPERVISION

Telegraf 1.33 (AMI) → InfluxDB 2.7 → Grafana 11.4

Deux minutes, un QR code, un appel.

“ Lundi, 8 h 47. Une nouvelle collaboratrice ouvre Asaphone sur son téléphone. Elle n'a ni compte, ni identifiants, ni contact avec l'administrateur : elle saisit son adresse e-mail, tape le code à six chiffres reçu dans sa boîte, puis scanne le QR qui arrive dans la foulée. À 8 h 49, son poste **1007** est enregistré sur le PBX en WebSocket sécurisé ; elle compose **1001** et son premier appel — chiffré de bout de jambe en bout de jambe — traverse le VLAN voix avec la priorité réseau d'un paquet marqué or. ”



Ce document raconte comment cette simplicité apparente est **fabriquée** : par un PBX durci, un réseau segmenté, une API de provisionnement embarquée et un démarrage entièrement automatisé.

Sommaire

PARTIE I — CADRAGE

01	Introduction.....	5
02	Problématique	5
03	Contexte d'entreprise et approche retenue	6

PARTIE II — CONCEPTION

04	Architecture générale.....	10
05	Réseau : adressage, VLAN voix et QoS.....	12
06	Sécurité en profondeur (L0 → L7).....	13

PARTIE III — RÉALISATION

07	Extensions et administration FreePBX.....	17
08	Services d'appel : IVR, files, conférences et groupes.....	19
09	WebRTC : WSS, DTLS-SRTP et pont média.....	21
10	Le client Asaphone	24
11	Messagerie vocale et chat.....	27
12	Provisionnement de bout en bout	28

PARTIE IV — EXPLOITATION

13	Accès distant : VPN WireGuard, tunnels et trunks	33
14	Supervision : Telegraf → InfluxDB → Grafana.....	35
15	Installation et démarrage automatisé	37
16	Validation et tests	41

PARTIE V — BILAN

17	Limites et perspectives.....	44
18	Conclusion.....	45

ANNEXES

A	Matrice des ports.....	46
B	Plan de numérotation complet.....	46
C	Glossaire	47
D	Table des figures.....	47

Cadrage

Avant la technique, le pourquoi : ce que coûte une téléphonie subie, ce qu'exige une téléphonie choisie, et la stratégie adoptée pour passer de l'une à l'autre.

01 Introduction

02 Problématique

03 Contexte d'entreprise et approche retenue

ASAPHONE
RAPPORT TECHNIQUE - 2026

01 Introduction

Un projet de téléphonie complet : du plan d'adressage au doigt qui décroche.

La téléphonie d'entreprise est un service que l'on remarque uniquement quand il tombe. Derrière chaque appel « qui marche », il y a un empilement précis : un réseau qui isole et priorise la voix, un cœur SIP qui route et ponte, du chiffrement à chaque segment, des identités provisionnées sans friction, et une exploitation qui survit aux redémarrages comme aux coupures Internet.

ASAPHONE couvre l'intégralité de cet empilement, avec deux composants jumeaux : d'un côté un **PBX Asterisk 20 LTS** administré par **FreePBX 17** sur une VM Ubuntu — dépôt `Asaph-D/serveur` — et de l'autre un **softphone Flutter multiplateforme** (Android, Windows, iOS) — dépôt `Asaph-D/asaphone`, projet OBSCURA. Le premier fournit le service ; le second le rend accessible sans qu'aucun utilisateur n'ait jamais à connaître un port, un codec ou un certificat.

› Ce que le lecteur va traverser

Partie	Question à laquelle elle répond	Chapitres
I — Cadrage	Pourquoi construire plutôt que louer ? Quelles contraintes structurent tout le reste ?	1 – 3
II — Conception	Comment l'architecture, le réseau et la sécurité sont-ils pensés <i>avant</i> la première extension ?	4 – 6
III — Réalisation	Comment les services concrets (postes, IVR, WebRTC, chat, provision) sont-ils construits ?	7 – 12
IV — Exploitation	Comment on y accède de loin, comment on l'observe, comment il démarre tout seul ?	13 – 16
V — Bilan	Qu'est-ce qui reste à faire, et qu'est-ce que le projet démontre ?	17 – 18

i CONVENTIONS DE LECTURE

Les illustrations sont numérotées en continu (Fig. 1, Fig. 2...) et récapitulées en annexe D. Les adresses IP montrées sont les valeurs d'exemple documentées (`pbx.local`, `10.10.10.10`, `192.168.1.80` ...) — l'IP LAN réelle varie par site et vit dans la configuration du serveur. Aucun secret n'est reproduit.

→ ET ENSUITE ?

La suite pose la question de départ : pourquoi ne pas simplement souscrire une offre cloud ? Le chapitre 2 formule la problématique qui justifie chaque choix technique de ce rapport.

02 Problématique

Quatre tensions à résoudre simultanément — et qui, prises séparément, ont des solutions contradictoires.

Formulée en une phrase, la question centrale du projet est la suivante :

QUESTION CENTRALE

Comment offrir à une petite structure une téléphonie **souveraine** (auto-hébergée, sans dépendance cloud), **sécurisée de bout en bout**, **utilisable par des non-techniciens en moins de deux minutes**, et **exploitable par une seule personne** — y compris quand Internet est coupé ?

Cette question se décompose en quatre tensions, chacune arbitrée dans ce rapport :

#	Tension	Pourquoi c'est difficile	Où c'est résolu
T1	Qualité vs mutualisation — la voix exige latence et priorité, le LAN bureautique n'en offre aucune	Un paquet RTP retardé de 150 ms s'entend ; un e-mail retardé de 2 s, non. Mélanger les deux trafics condamne la voix aux aléas du réseau data.	Ch. 5 — VLAN 10 dédié + DSCP EF
T2	Ouverture vs surface d'attaque — un PBX joignable de partout est un PBX attaqué de partout	Le SIP exposé est scanné en continu (force brute REGISTER). Mais le télétravail et les mobiles exigent un accès hors LAN.	Ch. 6 & 13 — UFW deny + Fail2Ban + WireGuard + tunnel sortant
T3	Simplicité vs rigueur — l'utilisateur veut un QR, la sécurité veut des secrets forts et révocables	Distribuer des mots de passe SIP de 16 caractères par oral ou par mail en clair ruine la politique de secrets.	Ch. 12 — provision QR one-shot, token révoqué au premier usage
T4	Automatisation vs fragilité — un démarrage à 17 étapes doit réussir même sans Internet	Tunnel Cloudflare, publication GitHub et VPN distant dépendent du réseau ; FreePBX, Apache et le LAN, non. Un boot monolithique casserait au premier lien coupé.	Ch. 15 — étapes critiques vs optionnelles (OK/SKIP)

À ces tensions s'ajoute une contrainte de méthode, assumée dès la phase 1 du dépôt : la modalité « **production d'abord** ». Aucune fonctionnalité n'est considérée acquise sur la foi d'une maquette — chaque brique est validée sur flux réel (appel effectif, capture réseau, requête API) et le résultat consigné. Cette exigence irrigue tout le document : les vérifications du chapitre 16 ne sont pas un appendice, elles sont la définition même du « terminé ».

→ ET ENSUITE ?

La problématique étant posée, le chapitre 3 la replace dans son contexte d'usage : quelle organisation, quels utilisateurs, et quelle stratégie pour tenir les quatre tensions à la fois.

03 Contexte d'entreprise et approche retenue

Une petite structure, un site principal, des collaborateurs mobiles — et le refus du SaaS par défaut.

› Le cadre d'usage

La plateforme est dimensionnée pour le profil d'organisation que dessinent les documents du dépôt : une **petite structure** (dix postes, pool d'extensions 1001–1010) avec un **site principal** équipé d'un LAN de gestion et d'un VLAN voix, et des **collaborateurs mobiles** — télétravail, déplacement, connexions 4G ou Starlink derrière du CGNAT. Trois populations cohabitent :

Population	Terminal	Besoin dominant
Postes fixes du site	Téléphones IP / softphones classiques (Zoiper) sur le VLAN voix ou le LAN	Fiabilité et qualité d'appel constantes

Population	Terminal	Besoin dominant
Collaborateurs équipés Asa-phone	Mobile ou desktop Flutter, sur le LAN ou à distance	Zéro configuration : s'inscrire, scanner, appeler
L'administrateur (unique)	UI FreePBX + SSH	Tout doit se réparer par script et survivre au reboot

› Pourquoi pas une offre cloud ?

Une solution SaaS aurait répondu en apparence plus vite. Elle a été écartée pour trois raisons convergentes : la **souveraineté des données** (les CDR, messages vocaux et annuaires restent sur une machine maîtrisée — MariaDB locale, spool local), le **coût récurrent par poste** qui pénalise les petites équipes, et surtout la **maîtrise du chemin critique** : ici, la téléphonie interne fonctionne intégralement *sans Internet* — le cœur PBX, le LAN et le VLAN voix sont autonomes, seuls les accès distants exigent le réseau extérieur. C'est une propriété qu'aucune offre hébergée ne peut fournir.

› L'approche : quatre phases, des scripts, pas de gestes manuels

Pour rendre le projet *faisable par une seule personne*, l'approche refuse le « clic unique dans une UI » comme méthode de déploiement. Chaque évolution du serveur est portée par un **script idempotent versionné** — rejouable sans dégât — et le cheminement suit quatre phases incrémentales, chacune adossée à un document d'exploitation :

Phase	Livrable	Ce qu'elle rend possible ensuite
1 — Réseau	VLAN 10 voix (10.10.10.0/24), QoS DSCP EF sur le RTP, localnets, UFW	Un socle où la voix est isolée et prioritaire — condition de tout le reste
2 — Utilisateurs	Extensions PJSIP 1001–1010, TLS 5061, messagerie vocale, groupe 8000	Des identités et des postes — la matière première des services
3 — Services	IVR AGI Python, files ACD, ConfBridge, monitoring Docker	L'expérience « standard d'accueil » et l'observabilité
4 — Sécurité	TLS généralisé, SRTP, Fail2Ban, durcissement UFW, politique de secrets	L'ouverture maîtrisée vers l'extérieur (VPN, provision, mobiles)

Au-delà des phases, trois principes transversaux structurent l'ensemble — on les retrouvera dans chaque chapitre :

- **Une seule source de vérité par domaine** — la configuration réseau du site vit dans `network/site.env` et `global-config.env` ; la configuration téléphonique vit en base FreePBX (jamais dans les fichiers générés) ; les secrets vivent hors dépôt (`/root/*.txt`, `/etc/provision/*.env`, `monitoring/.env`).
- **Le boot est le contrat** — tout ce qui est nécessaire au service est réappliqué à chaque démarrage par `serveur-startup.service` : permissions, certificats, pare-feu, tunnels, publication du bootstrap. Un serveur qui redémarre est un serveur réparé.
- **Le client ne sait rien, le serveur publie tout** — Asaphone découvre l'intégralité de sa configuration (API, WSS, VPN, codecs, salles de conférence) via un `bootstrap.json` publié ; aucune valeur n'est codée en dur côté client.

◆ L'ESSENTIEL DU CHAPITRE

- Cible : petite structure mono-site avec mobilité — 10 postes, 1 administrateur.
- Choix de l'auto-hébergement : souveraineté des données, coût, et téléphonie interne autonome sans Internet.
- Méthode : 4 phases incrémentales, scripts idempotents versionnés, secrets hors dépôt.
- Trois principes : source de vérité unique, boot auto-réparant, client entièrement provisionné.

→ ET ENSUITE ?

Le cadre est posé. La partie II ouvre le capot : d'abord la vue d'ensemble de l'architecture (chapitre 4), puis le réseau qui la porte (chapitre 5), enfin la sécurité qui l'enveloppe (chapitre 6).

ASAPHONE
RAPPORT TECHNIQUE · 2026

Conception

L'architecture avant le service : un cœur B2BUA, un réseau à trois pattes, et huit couches de sécurité emboîtées.

04 Architecture générale

05 Réseau : adressage, VLAN voix et QoS

06 Sécurité en profondeur (L0 → L7)

ASAPHONE
RAPPORT TECHNIQUE - 2026

04 Architecture générale

De l'Internet au combiné : chaque flux traverse une frontière de sécurité avant d'atteindre le cœur.

La lecture de l'architecture se fait du haut vers le bas : **en haut**, l'Internet — fournisseur SIP (phase trunk, préparée), tunnel Cloudflare pour l'API de provision distante, GitHub Pages pour la découverte, clients 4G. **Au milieu**, la frontière : UFW en *default deny*, Fail2Ban, et le point d'entrée WireGuard. **Au cœur**, la zone serveur : Asterisk et ses satellites (FreePBX, mini-API PHP, MariaDB, monitoring). **En bas**, les deux mondes desservis : le LAN de gestion et le VLAN voix.

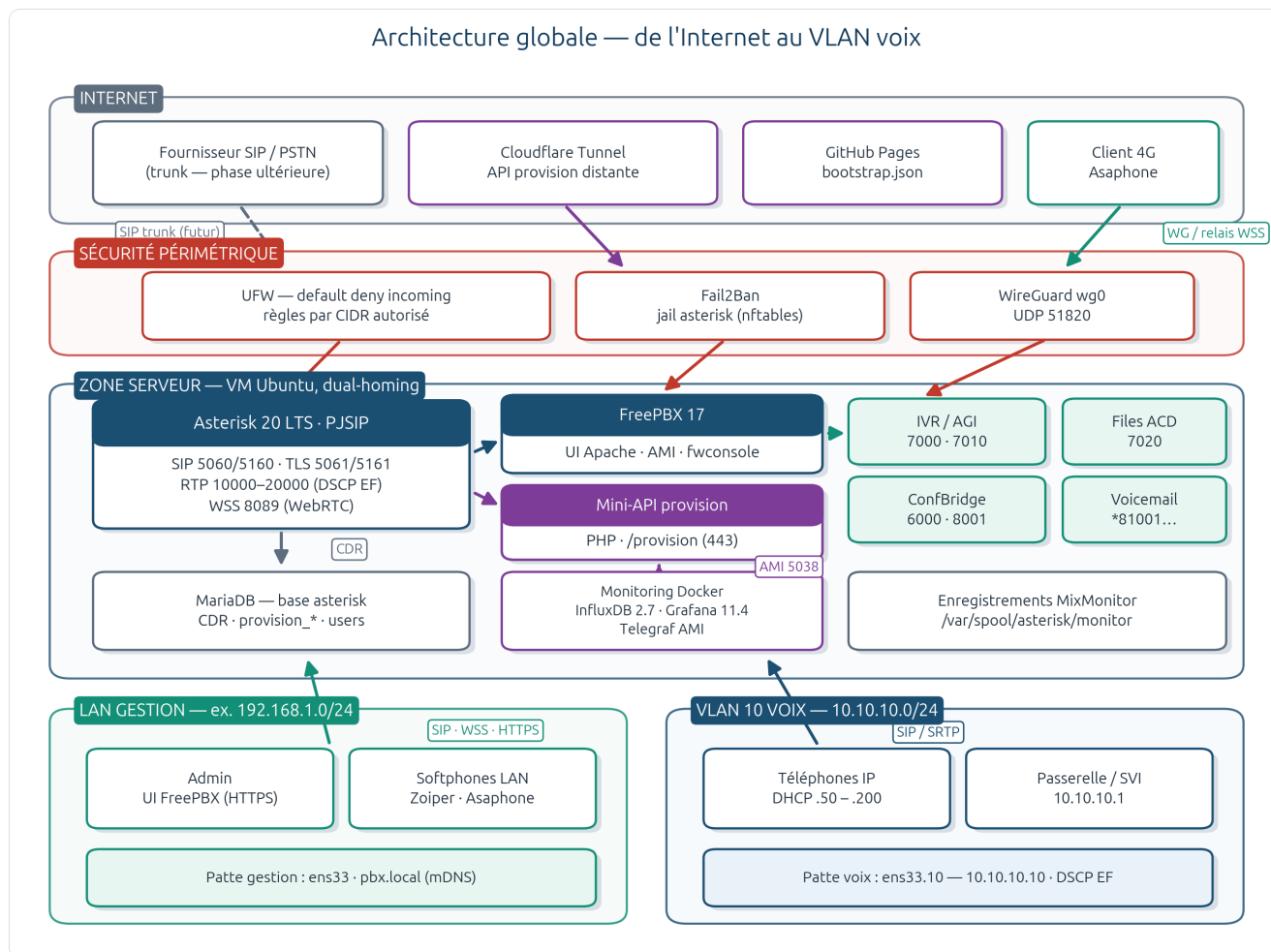


Fig. 1 · architecture globale — Internet → pare-feu → Asterisk/FreePBX → LAN gestion + VLAN voix

› Le choix structurant : un B2BUA au centre

Asterisk n'est pas un simple relais SIP : c'est un **B2BUA** (*back-to-back user agent*). Chaque appel est *terminé* sur le PBX puis *ré-émis* vers l'autre poste — deux jambes indépendantes, négociées séparément. Cette décision, qui peut sembler un détail d'implémentation, conditionne en réalité tout le projet :

- **Elle rend l'hétérogénéité possible** — un Zoiper en UDP/G.711 peut appeler un Asaphone en WSS/Opus : le PBX transcode et adapte l'enveloppe (chapitre 9).
- **Elle rend les services possibles** — ConfBridge mixe, MixMonitor enregistre, l'IVR interagit : autant d'opérations qui exigent l'accès au média en clair sur le PBX.
- **Elle fixe la limite du chiffrement** — chaque jambe est chiffrée (SRTP ou DTLS-SRTP), mais le PBX est un point de confiance : l'E2EE strict poste-à-poste est hors modèle (chapitre 6).

› Trois pattes réseau, trois mondes

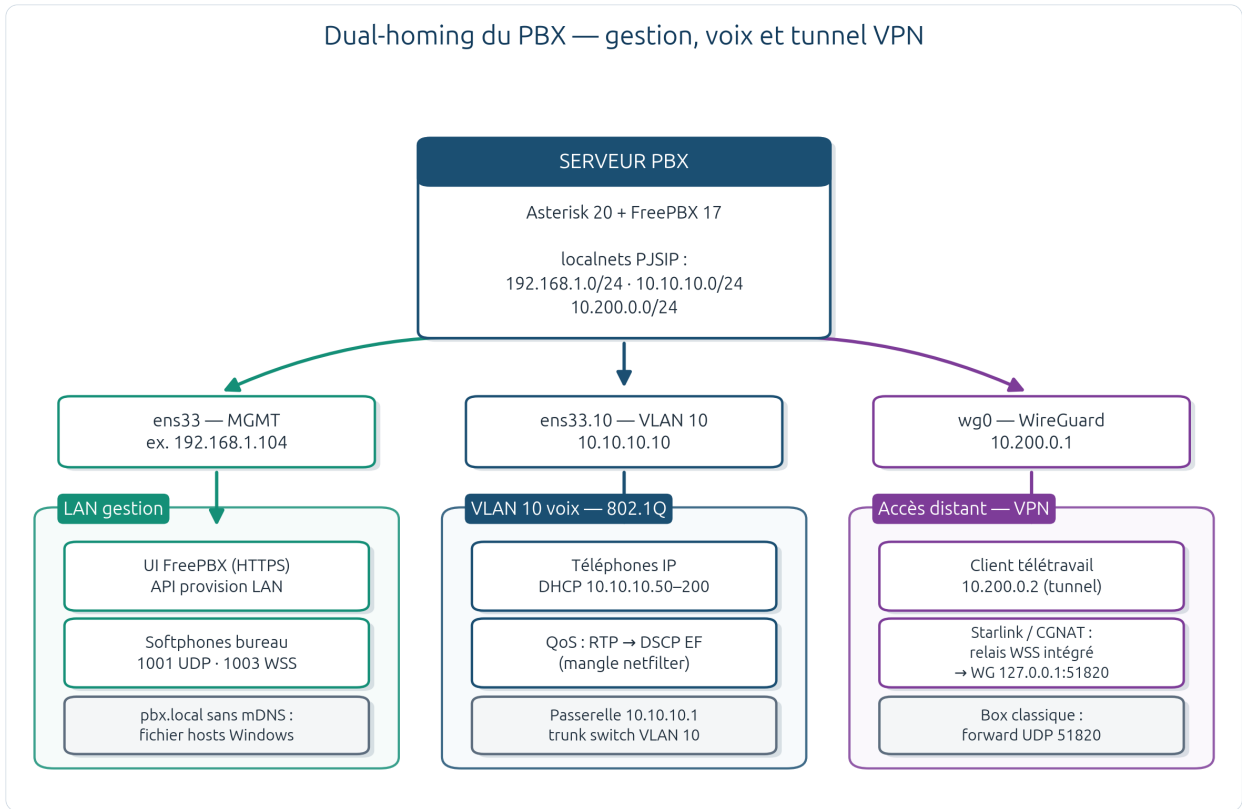


Fig. 2 · dual-homing du PBX — patte gestion (MGMT), patte voix (VLAN 10) et tunnel WireGuard, avec les clients de chaque monde

Interface	Réseau	Qui s'y connecte	Ce qui y circule
<code>ens33</code>	LAN gestion (ex. 192.168.1.0/24)	Admin (UI FreePBX), softphones bureau, API provision LAN	HTTPS 443, SIP 5060/5061, WSS 8089
<code>ens33.10</code>	VLAN 10 voix — 10.10.10.0/24	Téléphones IP du site (DHCP .50–.200)	SIP + RTP prioritaire (DSCP EF)
<code>wg0</code>	VPN — 10.200.0.0/24	Clients distants enrôlés (10.200.0.x)	Tout le trafic PBX, chiffré Wire-Guard

i POURQUOI LES LOCALNETS COMPTENT

PJSIP décide de réécrire (ou non) les adresses SIP selon que la source est « locale ». Les trois réseaux ci-dessus sont déclarés dans les **localnets** FreePBX (base `kvstore_Sipsettings`) et régénérés à chaque `fwconsole reload` — un poste VPN est ainsi traité comme un poste du LAN, sans bricolage NAT.

◆ L'ESSENTIEL DU CHAPITRE

- Lecture verticale : Internet → frontière (UFW/Fail2Ban/WG) → cœur (Asterisk + satellites) → LAN/VLAN.
- Le B2BUA est le choix fondateur : hétérogénéité des clients, services média, chiffrement par segment.
- Trois pattes réseau étanches ; les localnets PJSIP unifient leur traitement SIP.

→ ET ENSUITE ?

L'architecture suppose un réseau qui isole et priorise la voix. Le chapitre 5 détaille ce socle : plan d'adressage, VLAN 10 et marquage QoS.

05 Réseau : adressage, VLAN voix et QoS

Isoler d'abord, prioriser ensuite : le VLAN sépare les mondes, le DSCP fait passer la voix devant.

› Deux mécanismes distincts — et complémentaires

Le cahier « VLAN 10 dédié voix, QoS DSCP EF pour le RTP » recouvre deux choses que le document de phase 1 prend soin de distinguer, car elles ne se configurent pas au même endroit : le **VLAN** est un segment de couche 2 (802.1Q) — il se crée sur le switch et l'hyperviseur, Asterisk ne voit que des adresses IP ; le **marquage DSCP** est une étiquette de priorité posée sur chaque paquet IP — il se fait sur le serveur (netfilter) et doit être *honoré* par les équipements (*trust DSCP*).

Élément du plan	Valeur retenue	Justification
Réseau voix (VLAN 10)	10.10.10.0/24	Préfixe immédiatement lisible (« tout 10.10.10.x = voix »), aucun risque de collision avec les LAN 192.168.x des sites
PBX — patte voix	10.10.10.10/24 (statique, ens33.10)	IP stable pour les téléphones, connexion NetworkManager <code>voix-vlan10</code> , jamais de route par défaut
Passerelle voix	10.10.10.1	SVI/firewall — routage inter-VLAN contrôlé
Téléphones	DHCP 10.10.10.50 – 200	Plage .2–.49 réservée à l'infrastructure (SBC, SNMP...)
Plage RTP	UDP 10000 – 20000	Constatée dans <code>rtp_additional.conf</code> (FreePBX) — les règles QoS y sont alignées

Le choix d'un second /24 *distinct* plutôt qu'un réseau plat n'est pas esthétique : un même /24 posé sur deux VLANs casse ARP et routage ; un réseau plat unique renonce au filtrage inter-zones et mélange les domaines de broadcast. Le /24 dédié donne des ACL lisibles — « autoriser SIP/RTP depuis la zone voix » s'écrit en une règle UFW.

› La QoS en pratique

Le marquage est inséré dans la table `mangle` d'UFW (`/etc/ufw/before.rules`, sauvegarde préalable versionnée) : tout paquet UDP dont le port *source ou destination* tombe dans 10000–20000 reçoit la classe **EF (46, 0x2e)** — la file « expedited forwarding » que les équipements réseau servent en premier. La vérification fait partie de la procédure :

```
$ sudo iptables -t mangle -L OUTPUT -n -v
DSCP set 0x2e udp multiport sports 10000:20000
DSCP set 0x2e udp multiport dports 10000:20000
# contrôle en appel réel : tcpdump -vv -n -i ens33.10 udp portrange 10000-20000 → champ tos/DSCP
```


⚠ **FRONTIÈRE DE RESPONSABILITÉ**

Côté serveur, tout est en place (interface, localnets, UFW, marquage). Le raccordement physique — trunk 802.1Q sur le switch, port group VLAN 10 sur l'hyperviseur, files prioritaires *trust DSCP* — relève de l'infrastructure du site et est documenté comme restant à réaliser (Plan-adressage §8). La VM est prête : dès que le lien tagué arrive, `ens33.10` est opérationnelle.

◆ **L'ESSENTIEL DU CHAPITRE**

- VLAN (L2, switch/hyperviseur) et DSCP (L3, serveur) sont deux mécanismes distincts, tous deux nécessaires.
- Plan : 10.10.10.0/24, PBX en .10, passerelle .1, téléphones .50–.200, RTP 10000–20000.
- Marquage EF posé dans `ufw/before.rules`, aligné sur la plage RTP FreePBX, vérifiable en une commande.

→ **ET ENSUITE ?**

Un réseau isolé n'est pas encore un réseau sûr. Le chapitre 6 empile les huit couches de sécurité qui enveloppent ce socle — du tag VLAN au stockage des secrets.

06 Sécurité en profondeur (L0 → L7)

Aucun mécanisme unique ne protège un PBX : huit couches se recouvrent, chacune rattrapant les failles de la précédente.

La sécurité du projet n'est pas un chapitre ajouté à la fin — c'est la **phase 4** du déploiement, et elle est pensée en couches emboîtées. Chaque couche a son mécanisme, son script d'application et son contrôle. Le schéma suivant en donne l'état documenté :

Sécurité en profondeur — huit couches emboîtées, de la trame au secret		
L0	Segmentation L2 VLAN 10 voix (10.10.10.0/24) séparé du LAN gestion — 802.1Q, QoS trust DSCP	déployé
L1	Transport réseau VPN WireGuard wg0 (10.200.0.0/24, UDP 51820) — relais WSS si CGNAT	déployé
L2	Pare-feu / anti-abus UFW default deny + règles par CIDR · Fail2Ban jail asterisk	déployé
L3	Administration web HTTPS Apache 443 (certificat) · session FreePBX · permissions GUI	déployé
L4	Signalisation SIP TLS 5061/5161 · WSS 8089 · authentification Digest	partiel
L5	Média audio / vidéo SRTP SDES (postes classiques) · DTLS-SRTP (WebRTC)	partiel
L6	Provisionnement QR chiffré one-shot · claim 24 h · révocation jti · rate-limit	déployé
L7	Données au repos Secrets 600/640 hors Git · rotation 90 j · CDR MariaDB	partiel

Chaque jambe est chiffrée ; le PBX reste un point de confiance (B2BUA) — l'E2EE strict poste-à-poste est hors modèle.

Fig. 3 · matrice des couches de sécurité L0 → L7 et leur état

› Signalisation et média : chiffrer chaque jambe

Segment	Mécanisme	Mise en œuvre	Contrôle
SIP classique	TLS 1.2+ sur 5061	Certificat Certman assigné à PJSIP (<code>pjsipcert-tid</code>) par <code>phase4-assign-pjsip-tls-cert.php</code>	<code>pjsip show transport 0.0.0.0-tls</code> → <code>cert_file</code> renseigné
Média classique	SRTP SDES	<code>media_encryption=sdes</code> sur 1001–1010 (<code>phase4-enable-srtp-extensions.php</code>)	<code>grep media_encryption /etc/asterisk/pjsip.endpoint.conf</code>
SIP WebRTC	WSS (TLS 8089)	Transport <code>transport-wss</code> + certificats Certman (<code>fix-wss-tls.sh</code>)	<code>http show status</code> → HTTPS 8089
Média WebRTC	DTLS-SRTP + ICE	Profil endpoint WebRTC (<code>align-pjsip-site.sh</code>); module <code>res_srtp</code> requis	Appel réel 1001 ↔ 1003 (ch. 16)

› Frontière : refuser par défaut, bannir les insistants

- **UFW** — politique `default deny incoming` ; le SIP et le RTP ne sont ouverts que depuis les CIDR déclarés (VLAN voix, LAN gestion, VPN). Durcissement notable de la phase 4 : **5061/tcp n'est plus exposé « Anywhere »** — il est restreint aux réseaux nommés ; les futurs trunks opérateur passeront par des règles ciblées par IP du fournisseur.
- **Fail2Ban** — jail `asterisk` sur `/var/log/asterisk/full` (backend auto, ports 5060/5061/5160/5161) : bannissement /32 après échecs répétés. Le filtre se teste sur un extrait du log avec `phase4-test-fail2ban-filter.sh` (le log complet rendrait `fail2ban-regex` interminable).
- **Secrets** — mots de passe SIP aléatoires ≥ 16 caractères, rotation planifiée à 90 jours ; identifiants MariaDB uniquement dans `/etc/freepbx.conf` ; aucun secret dans Git.

› Le point souvent oublié : les permissions

⚠ LEÇON D'EXPLOITATION — LES CLÉS TLS ET LE GROUPE ASTERISK

Un incident documenté (`network/run.txt`) : des clés en `0600 www-data:www-data` **cassent le WSS**, car le démon Asterisk ne lit qu'avec le groupe `asterisk`. Le modèle durable est `www-data:asterisk` en `0640`, réappliqué à chaque boot par `freepbx_chown.conf` + les scripts `fix-*` — jamais de `chmod 777`, jamais de `chown -R` global.

i LA LIMITE ASSUMÉE : PAS D'E2EE STRICT

Chaque jambe est chiffrée, mais le PBX déchiffre pour ponter, transcoder, mixer (ConfBridge) et enregistrer (MixMonitor). Le modèle de menace retenu fait du serveur un **point de confiance** — c'est le prix des services média, et c'est documenté plutôt que masqué.

◆ L'ESSENTIEL DU CHAPITRE

- Huit couches emboîtées, chacune scriptée et contrôlable — la phase 4 est un durcissement, pas un vernis.
- Chiffrement par segment : TLS/SRTP côté classique, WSS/DTLS côté WebRTC ; le PBX reste point de confiance.
- 5061 fermé au monde ; Fail2Ban banni les scans ; secrets ≥ 16 caractères hors dépôt, rotation 90 j.
- Les permissions (clés 0640 `www-data:asterisk`) sont une couche de sécurité à part entière, réappliquée au boot.

→ ET ENSUITE ?

La conception est complète : architecture, réseau, sécurité. La partie III construit dessus, en commençant par la matière première de toute téléphonie — les extensions (chapitre 7).

ASAPHONE
RAPPORT TECHNIQUE · 2026

Réalisation

Des identités aux services : postes PJSIP, standard d'accueil, WebRTC, client Flutter, messagerie et provisionnement automatisé.

- 07 Extensions et administration FreePBX
- 08 Services d'appel : IVR, files, conférences et groupes
- 09 WebRTC : WSS, DTLS-SRTP et pont média
- 10 Le client Asaphone
- 11 Messagerie vocale et chat
- 12 Provisionnement de bout en bout

07 Extensions et administration FreePBX

Dix postes, deux profils média, zéro édition manuelle de fichier : tout passe par la base.

› La règle d'or FreePBX

Sous FreePBX, les fichiers Asterisk « classiques » (`pjsip.conf`, `extensions.conf`...) sont **générés** : les modifier à la main, c'est écrire dans du sable — le prochain `fwconsole reload` les écrase. Le flux légitime est :

```
UI FreePBX (ou scripts PHP du dépôt) → base MariaDB (asterisk) → fwconsole reload → /etc/asterisk/*.conf (générés)
Custom autorisé : extensions_custom.conf · pjsip_custom_post.conf · queues_custom.conf · manager_custom.conf
```

C'est pourquoi la phase 2 crée les postes par **script PHP FreePBX** (`phase2-create-extensions.php`) plutôt qu'à la main : contexte `from-internal`, **3 contacts max** par extension (plusieurs appareils simultanés), secrets aléatoires écrits dans `/root/phase2-pjsip-secrets.txt` (`chmod 600`, hors Git), boîte vocale activée pour chacun.

Extensions	Profil	Transport	Média	Codecs
1001 – 1002	Classique (Zoiper, téléphone IP)	UDP 5060 · TLS 5061	RTP + SRTP SDES	ulaw · alaw · gsm · g726 · g722
1003 – 1010	WebRTC (Asaphone) — pool de provision	WSS 8089	DTLS-SRTP · ICE · AVPF · rtcp-mux	Opus + ulaw/alaw
8000	Sonnerie de groupe (dialplan custom)	—	Dial simultané 1001... 1010, 45 s	—

› L'interface au quotidien

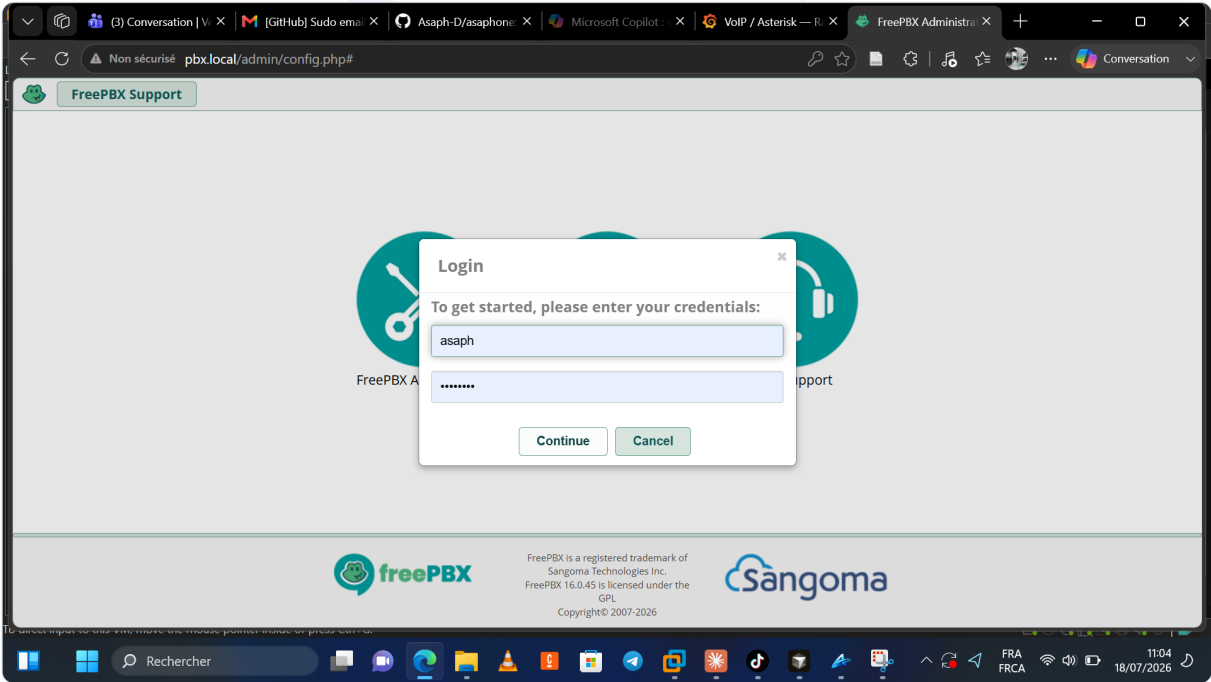


Fig. 4 · tableau de bord FreePBX 17 — l'UI d'administration du PBX

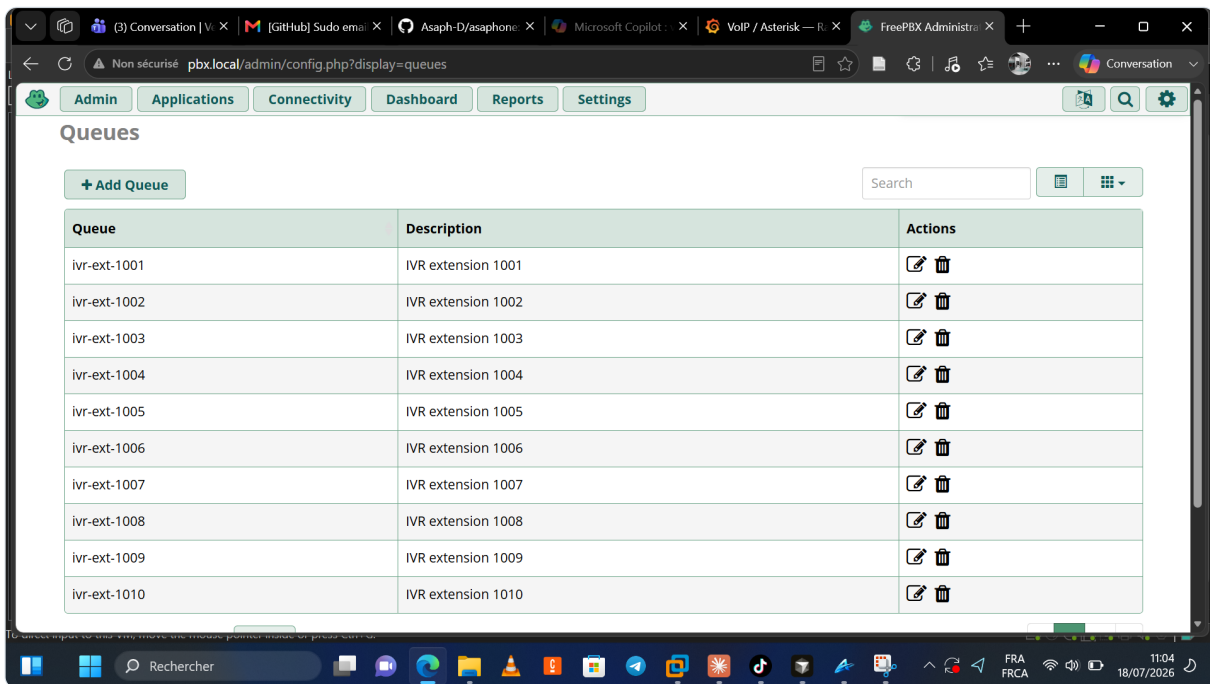


Fig. 5 · liste des extensions PJSIP dans FreePBX (Applications → Extensions)

› Créer un poste : le parcours complet

Le formulaire de création (ci-dessous) illustre le principe base-d'abord : les champs saisis alimentent MariaDB, et c'est le *reload* qui matérialise le poste dans la configuration Asterisk. Les groupes d'appel (`namedcallgroup` / `namedpickupgroup` « phase2 ») sont posés sur les dix postes ; la sonnerie générale répond au **8000**.

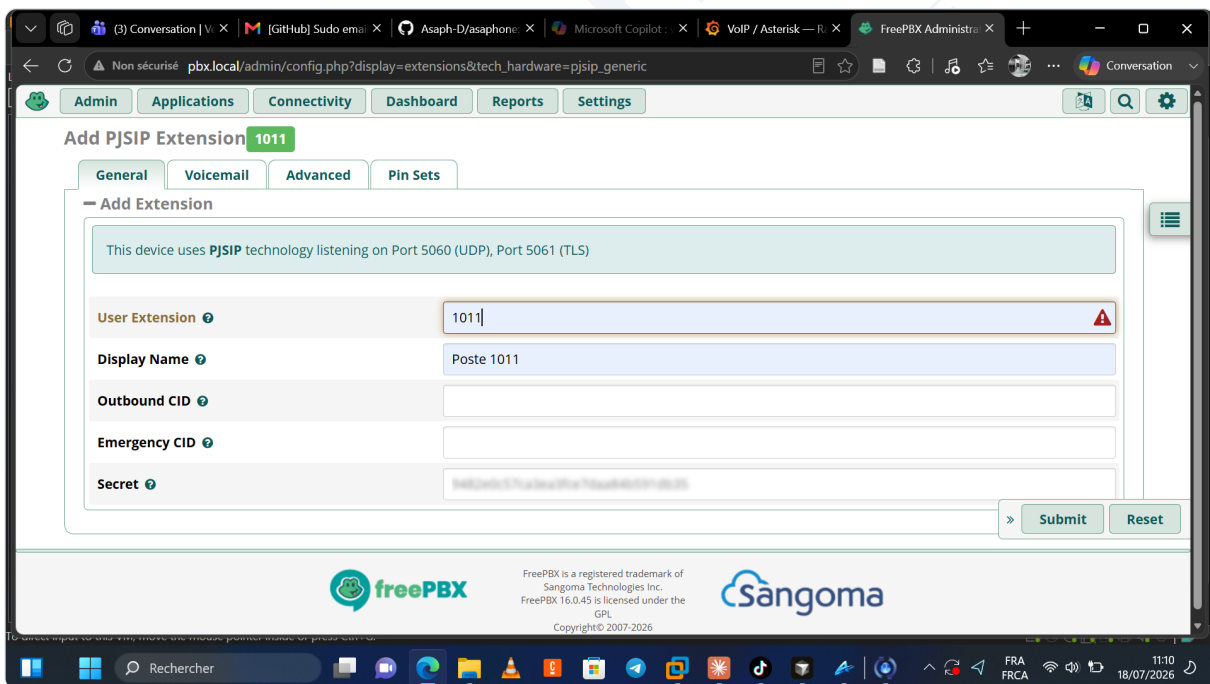


Fig. 6 · création d'une nouvelle extension PJSIP

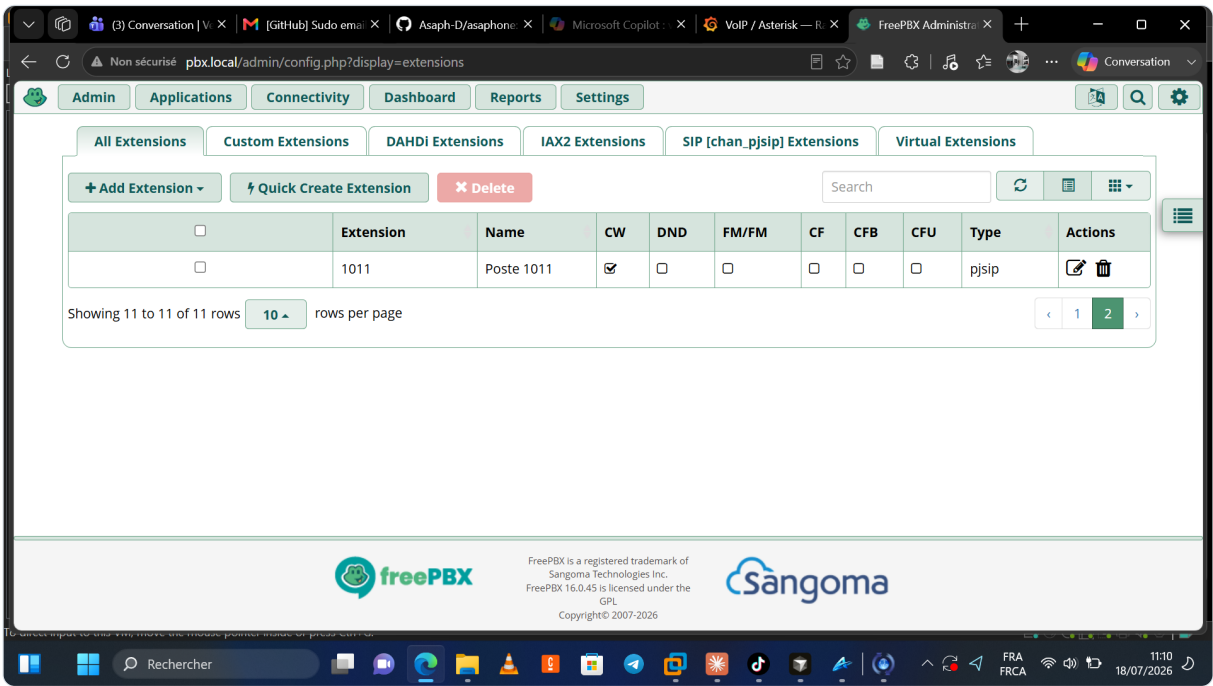


Fig. 7 · l'extension créée, prête à être rechargée

◆ L'ESSENTIEL DU CHAPITRE

- Jamais d'édition des fichiers générés : UI/scripts → MariaDB → fwconsole reload.
- 10 extensions scriptées, secrets ≥ 16 caractères hors dépôt, 3 appareils par poste.
- Deux profils média distincts alignés par align-pjsip-site.sh ; sonnerie de groupe 8000 en custom.

→ ET ENSUITE ?

Des postes qui sonnent, c'est le minimum. Le chapitre 8 leur ajoute un standard : accueil horaire, IVR intelligent, files d'attente et salles de conférence.

08

Services d'appel : IVR, files, conférences et groupes

Un standard d'accueil complet — porté par le dialplan custom, indépendant des modules GUI.

› Le parcours d'un appel entrant

Le service d'accueil suit une chaîne à trois étages, chacun étant un numéro interne testable isolément : le **7000** applique la fenêtre horaire (`GotoIfTime`, lun-ven 9h-18h) et route vers le **7010** — l'IVR proprement dit, un **AGI Python** (`phase3_intelligent_ivr.py`) qui reconnaît les appelants VIP (`phase3-vip.txt`), adapte son comportement à l'heure et permet la saisie directe d'une extension ; en bout de chaîne, le **7020** place l'appelant dans la file `phase3-support` (stratégie *leastrecent*, membres 1001-1010) dont les conversations sont enregistrées par Mix-Monitor.

Numéro	Service	Détail
7000	Porte d'entrée horaire	Ouvré → 7010 ; fermé → message de fermeture
7010	IVR AGI Python	VIP, logique horaire/langue, saisie d'extension
7020	File ACD support	leastrecent, 1001-1010, MixMonitor → /var/spool/asterisk/monitor/

Numéro	Service	Détail
7101-7110	Files individuelles	ivr-ext-1001 ... ivr-ext-1010 (un poste chacune)
8001	ConfBridge à PIN	Salle phase3-<PIN> (PIN de test à changer)
8100	Test d'enregistrement	MixMonitor + renvoi messagerie 1001

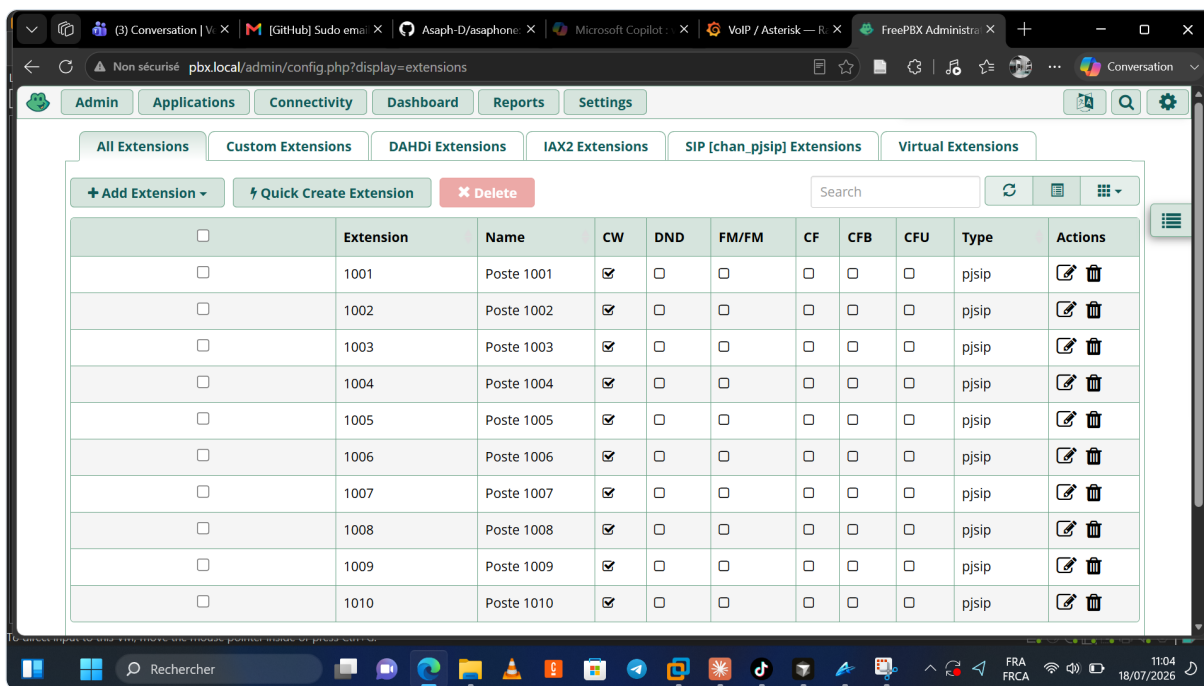


Fig. 8 · IVR et files d'attente couvrant toutes les extensions (phase 3)

i RÉSILIENCE VIS-À-VIS DES MODULES SANGOMA

Le téléchargement des gros modules GUI (queues, packs de sons) échouait en timeout sur le miroir Sangoma. Plutôt que d'attendre, la phase 3 s'appuie directement sur `app_queue` et `ConfBridge` via les fichiers `*_custom.conf`, appliqués par blocs balisés (`BEGIN_PHASE3...END_PHASE3`) et donc rejouables. Les modules GUI restent installables plus tard.

› Appels de groupe Asaphone : le client ne mixe jamais

Le principe est radical de simplicité côté téléphone : pour appeler un groupe, Asaphone compose **un seul numéro** (`call_uri`). Tout le reste est serveur — le dialplan fait entrer l'appelant dans une salle **ConfBridge**, puis le PBX *originate* vers chaque membre. Les groupes créés dans l'application sont synchronisés (`groups/sync.php`) et reçoivent chacun une salle dédiée `asaphone-grp-<slog>` ; la salle **6000** sert de défaut, les **6001-6099** sont réservées.

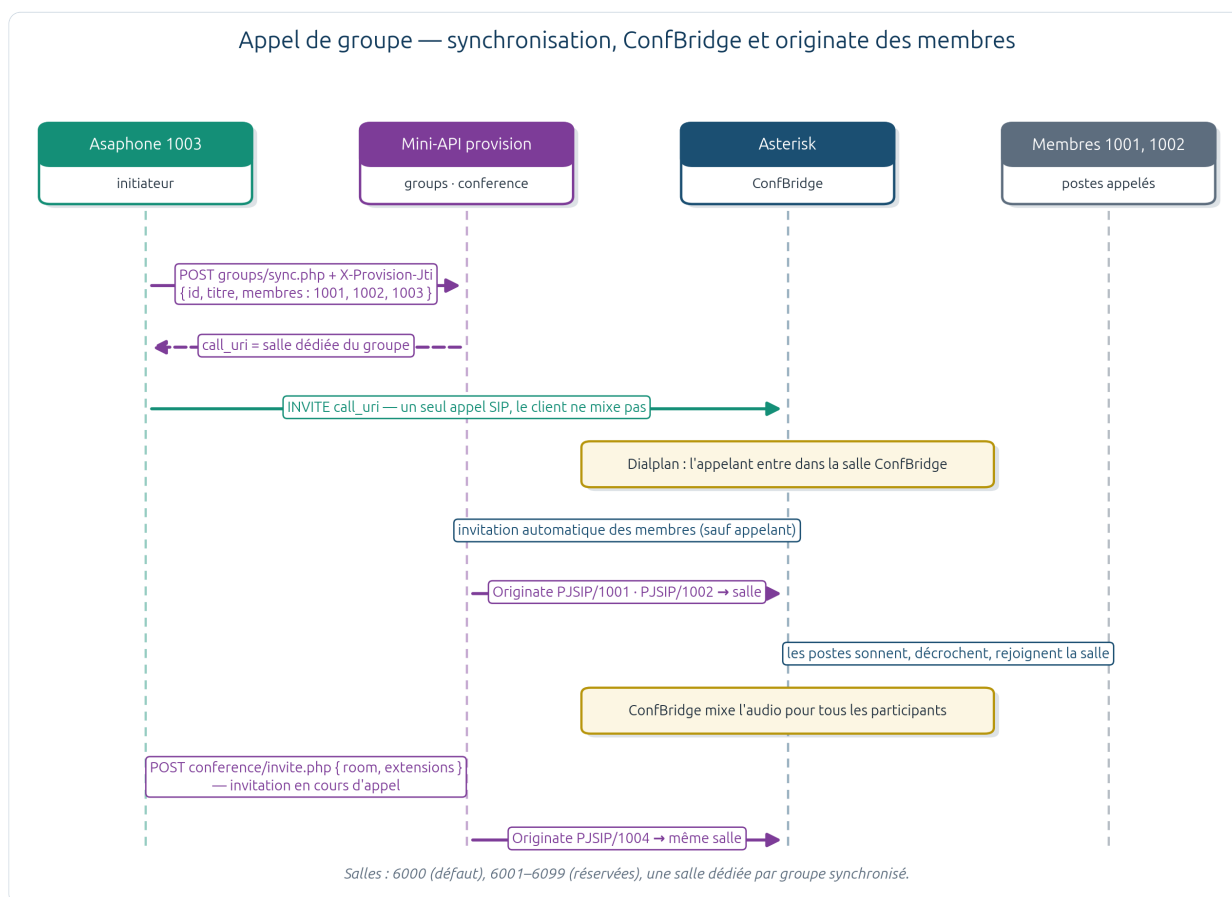


Fig. 9 · appel de groupe — synchronisation, ConfBridge et originate des membres

En cours d'appel, l'invitation d'un participant supplémentaire passe par `POST conference/invite.php` (extensions listées, ou `auto:true` pour tout le groupe) : le client reste dans la salle, le PBX fait sonner les invités. L'authentification de ces API suit le modèle commun du projet — paramètre `?ext=` + en-tête `X-Provision-Jti`.

◆ L'ESSENTIEL DU CHAPITRE

- Chaîne d'accueil 7000 → 7010 (AGI Python, VIP) → 7020 (ACD enregistrée) — testable étage par étage.
- Dialplan custom balisé et rejouable, indépendant des modules GUI Sangoma.
- Groupes : un seul INVITE client ; ConfBridge mixe, le PBX originaite les membres, invitation à chaud par API.

→ ET ENSUITE ?

Tous ces services supposent que des clients très différents — téléphone SIP et application WebRTC — puissent converser. Le chapitre 9 explique comment le PBX réconcilie ces deux mondes.

Faire parler un navigateur avec un téléphone SIP : deux dialectes, un traducteur central.

› Le problème à résoudre

Un client WebRTC et un softphone SIP classique ne parlent pas la même langue média. Le premier exige **DTLS-SRTP**, **ICE**, le profil **SAVPF** et privilège **Opus** ; le second envoie du RTP/AVP en G.711 sans enveloppe particulière. Les faire dialoguer « en direct » est impossible — et c'est exactement ce que le modèle B2BUA du chapitre 4 résout : Asterisk termine la jambe WebRTC d'un côté, ouvre une jambe RTP classique de l'autre, et traduit au milieu.

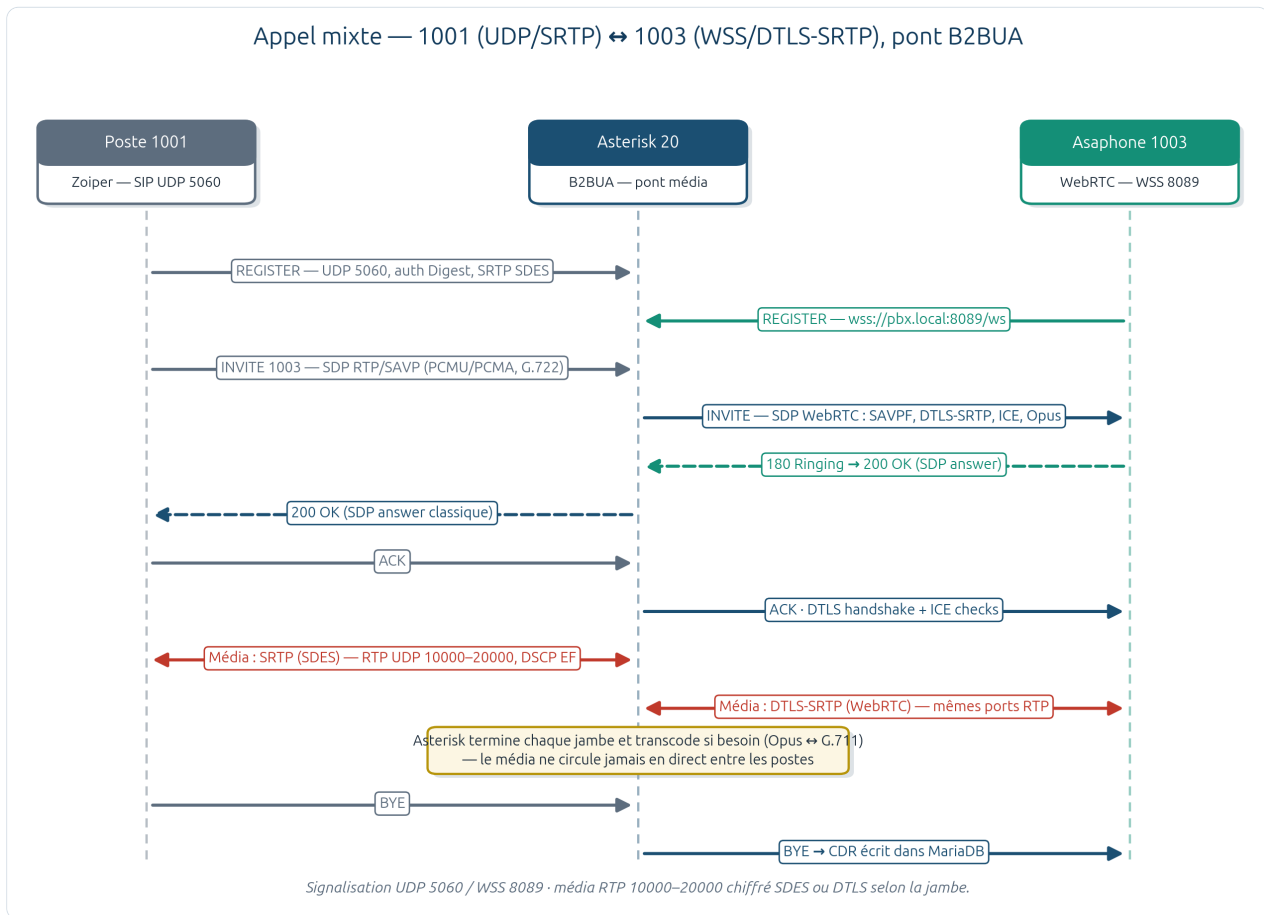


Fig. 10 · appel 1001 (UDP/SRTP) ↔ 1003 (WSS/DTLS-SRTP) — deux jambes négociées séparément, pont sur le PBX

› Côté serveur : ce qui rend le WSS possible

Brique	Réglage	Piège documenté
Mini-serveur HTTP Asterisk	TLS 8089 (WSS), 8088 en lab ; URL <code>wss://pbx.local:8089/ws</code>	FreePBX régénère <code>http_additional.conf</code> avec bind 127.0.0.1 → le script force <code>HTTP_TLS_BINDADDRESS</code> via <code>fwconsole setting</code>
Transport PJSIP	<code>transport-wss</code> lié à 0.0.0.0:8089	Vérifier par <code>pjsip show transports</code>
Module SRTP	<code>res_srtp.so</code> chargé	Absent d'une compilation sans <code>libsrt2</code> → 488 systématique ; script <code>install-asterisk-res-srtp.sh</code>
Endpoint WebRTC	<code>media_encryption=dtls</code> , ICE, AVPF, <code>rtcp_mux</code> , Opus+G.711	Sans ce profil, la signalisation WSS passe mais le média échoue (488 / « couldn't negotiate stream »)
Certificats	Certman <code>default.crt/key</code> , clés lisibles par le groupe asterisk	Perms 0600 → WSS ne bind pas (cf. ch. 6)

› Côté client : la configuration se réduit à trois champs

Grâce au provisionnement (chapitre 12), l'utilisateur ne voit jamais ces réglages — mais ils existent, et l'écran de configuration SIP d'Asaphone les expose pour le diagnostic : serveur (`pbx.local`), extension, transport WSS.

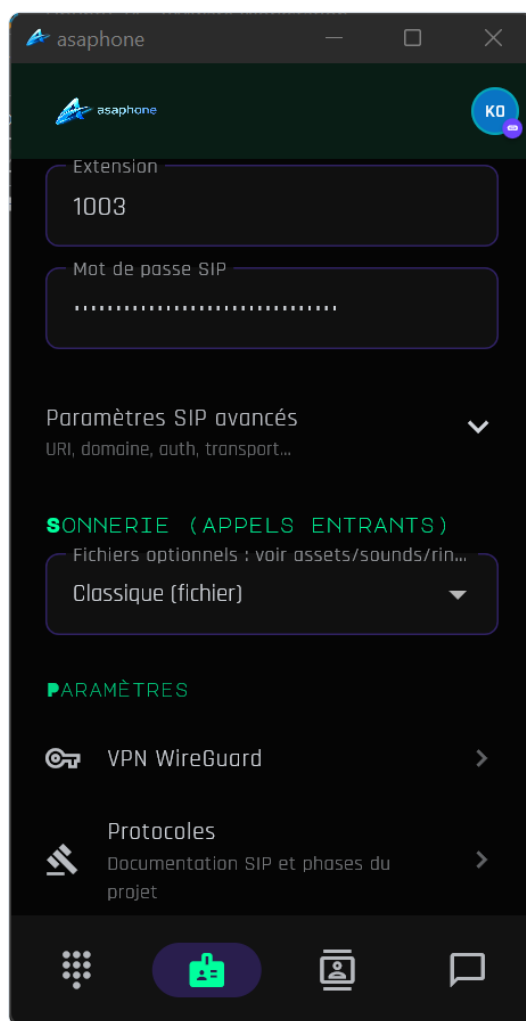


Fig. 11 · configuration SIP du client Asaphone — serveur, extension, transport WSS

⚠ DEUX COMPORTEMENTS CLIENT À CONNAÎTRE

1. Un REGISTER avec **Contact: *** et **Expires: 0** désenregistre *tous* les contacts du poste : à réserver au logout explicite, sous peine de voir le poste « Unreachable » et les appels filer en messagerie. 2. L'avertissement « DTLS packet dropped, ICE not completed yet » observé en lab est un comportement ICE côté client — l'appel s'établit, mais c'est le premier endroit où chercher si l'audio devient instable.

◆ L'ESSENTIEL DU CHAPITRE

- `wss://pbx.local:8089/ws` : le chemin `/ws` est celui d'Asterisk, le certificat doit couvrir le nom utilisé.
- Un endpoint WebRTC sans `dtls/ice/avpf/rtp-mux` échoue en 488 même si le WSS fonctionne.
- Aligner Opus + G.711 des deux côtés évite le transcodage CPU sur le pont.

→ ET ENSUITE ?

Le transport est prêt ; reste à voir ce que l'utilisateur tient en main. Le chapitre 10 parcourt le client Asaphone écran par écran.

10 Le client Asaphone

Un seul code Flutter pour Android, Windows et iOS — et un client qui ne connaît rien d'avance.

Asaphone est développé en **Flutter** (projet OBSCURA), ce qui donne un client unique pour les trois plateformes cibles. Son principe directeur rejoint celui du chapitre 3 : *le client ne sait rien, le serveur publie tout*. Au démarrage, l'application récupère le `bootstrap.json` (découverte), puis sa session provisionnée lui fournit — outre les credentials SIP — les endpoints d'API, les salles de conférence, les groupes, les codecs vidéo (VP8/H.264) et les serveurs ICE. Le diagramme suivant situe tout ce qu'un utilisateur peut faire, et par quel canal :

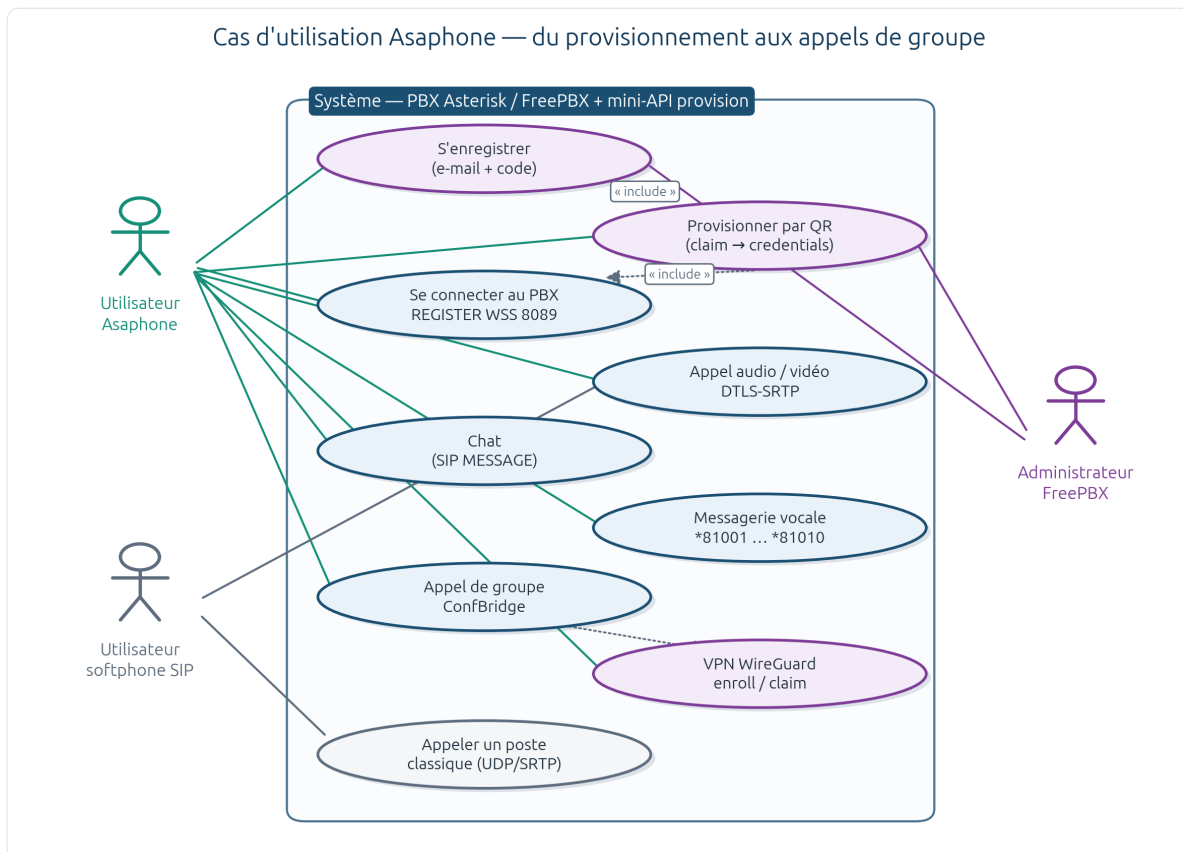


Fig. 12 · cas d'utilisation Asaphone ↔ PBX — provision, appels, chat, messagerie, groupes, VPN

› Premier contact : accueil et configuration

L'écran d'accueil propose les deux parcours du flux d'onboarding (« J'ai déjà mes identifiants » / « M'enregistrer ») ; la page de configuration récapitule le compte et l'état de l'enregistrement SIP.

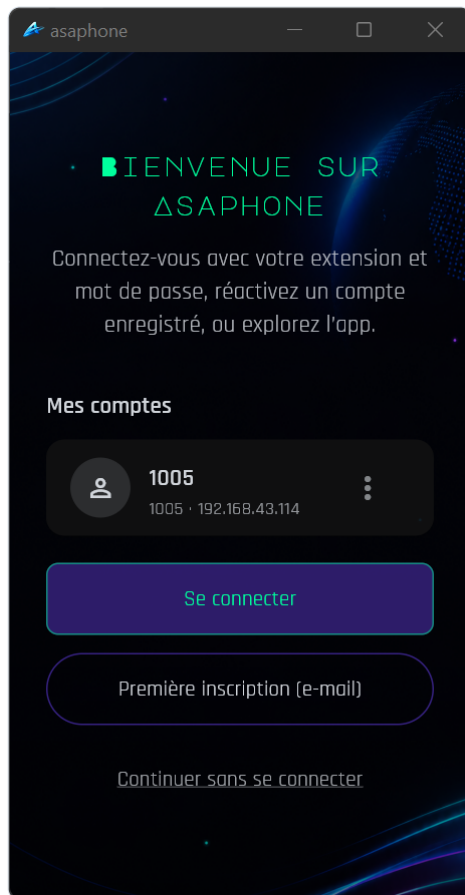


Fig. 13 · écran d'accueil / connexion d'Asaphone

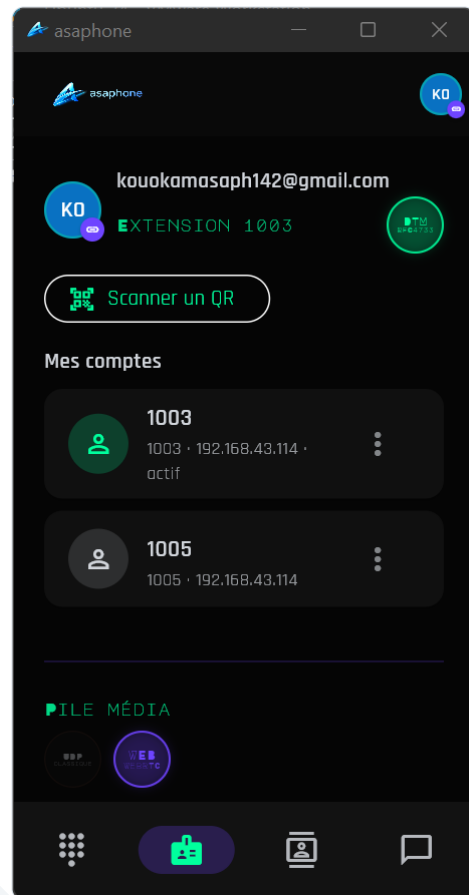


Fig. 14 · page de configuration du client

› Le geste quotidien : composer, parler, se voir

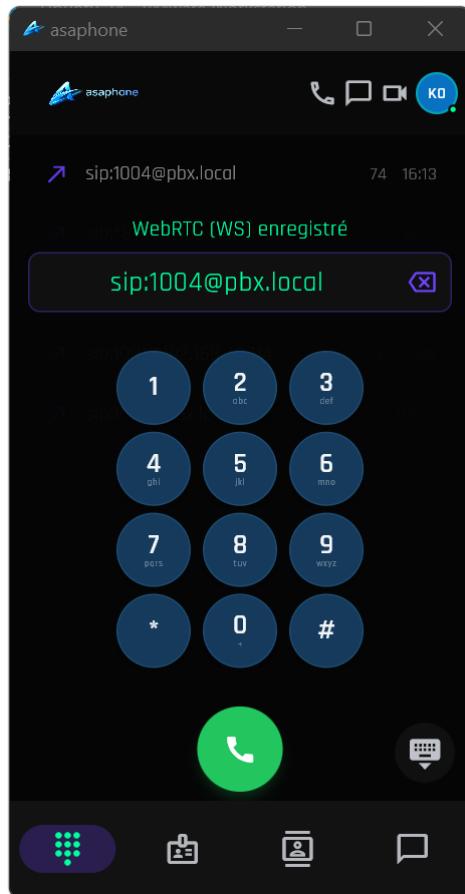


Fig. 15 · clavier d'appel (numérotation interne)

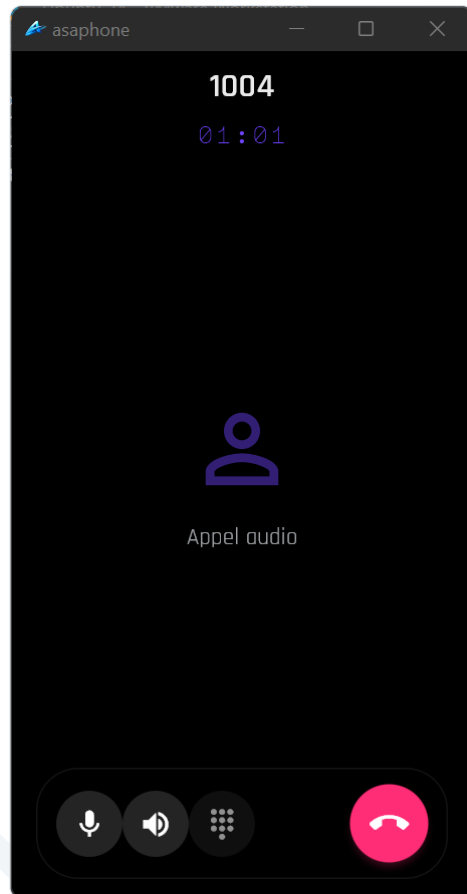


Fig. 16 · appel audio en cours (démonstration)

La vidéo emprunte exactement le même chemin que l'audio — SDP négocié via WSS, média DTLS-SRTP — avec les codecs annoncés par le bootstrap (**VP8, H.264**) :

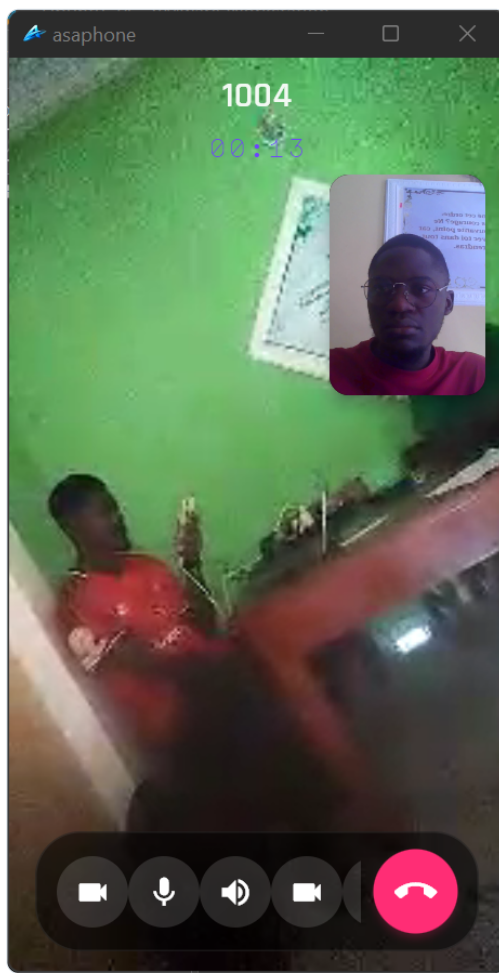


Fig. 17 · appel vidéo en cours (démonstration)

◆ L'ESSENTIEL DU CHAPITRE

- Un code Flutter, trois plateformes ; zéro constante serveur en dur dans l'application.
- Bootstrap → session : credentials, endpoints, groupes, conférences, ICE et codecs arrivent du PBX.
- Audio et vidéo partagent la même pile WSS + DTLS-SRTP (VP8/H.264 pour la vidéo).

→ ET ENSUITE ?

Au-delà de l'appel en direct, il faut gérer l'asynchrone : messages vocaux et texte. C'est l'objet du chapitre 11.

11 Messagerie vocale et chat

L'asynchrone sans serveur supplémentaire : app_voicemail pour la voix, SIP MESSAGE pour le texte.

› Messagerie vocale : dix boîtes, un incident instructif

Chaque poste 1001–1010 dispose d'une boîte vocale (contexte `default`) créée et mappée par `phase2-enable-voicemail.php` : pièce jointe WAV (`attach=yes`), horodatage et identification de l'appelant annoncés, PIN initial à personnaliser. L'incident documenté du dépôt mérite d'être raconté : certains appels basculaient en « an error has occurred » — le log révélait `No entry in voicemail config file for '1001'`. Diagnostic : l'extension existait côté PJSIP mais *pas sa mailbox* côté Voicemail. La remise en cohérence par script (plutôt qu'un correctif manuel par poste) a rétabli les dix boîtes d'un coup — et illustré la valeur de l'approche scriptée.

Fonction	Mise en œuvre
Consultation directe	Codes *81001 ... *81010 → boîte du poste correspondant (<code>apply-voicemail-codes.sh</code>)
Dépôt de message	Automatique après le bip (renvoi sur non-réponse / occupation / poste injoignable)
Notification e-mail	WAV en pièce jointe ; SMTP + adresse à renseigner par poste dans l'UI (reste à faire, S2 §7)
Notification Asaphone	<code>asaphone-vm-notify.sh</code> — notification poussée au client via le flux provision
Contrôle	<code>asterisk -rx "voicemail show users"</code> → default 1001 ... 1010

› Chat : le texte voyage dans la signalisation

Plutôt que d'ajouter un serveur XMPP ou un service de messagerie tiers, le chat d'Asaphone réutilise le canal déjà authentifié et chiffré : des requêtes **SIP MESSAGE** hors dialogue, routées par un bloc dédié du dialplan (`apply-message-dialplan.sh` , marqueurs `BEGIN_SIP_MESSAGE...END`). Les bénéfices sont immédiats : **une seule identité** (les credentials SIP servent à tout), **un seul chemin réseau** (le WSS déjà ouvert), et une intégration naturelle avec les notifications de messagerie vocale qui empruntent le même flux. Côté serveur, `asaphone-chat-ingest.sh` archive les messages en base (`provision-schema-chat.sql`), ce qui permet au client de resynchroniser son historique via l'API provision après réinstallation.

◆ L'ESSENTIEL DU CHAPITRE

- Dix boîtes vocales scriptées ; codes *8100X ; l'incident « mailbox manquante » a validé l'approche scriptée.
- Chat = SIP MESSAGE dans le canal SIP existant — pas de serveur de messagerie supplémentaire.
- Historique ingéré en base → resynchronisation du client via l'API provision.

→ ET ENSUITE ?

Voix, vidéo, texte, groupes : tous ces services supposent un compte. Le chapitre 12 montre comment ce compte naît — en deux minutes, sans administrateur.

12 Provisionnement de bout en bout

De l'e-mail au REGISTER : la scène d'ouverture du rapport, décomposée image par image.

› Trois étages de découverte

Le défi initial est trivial en apparence : *comment une application fraîchement installée sait-elle où est son serveur ?* La réponse tient en trois étages, du plus stable au plus volatil :

Étage	Support	Contenu	Disponibilité
Découverte	GitHub Pages — <code>bootstrap.json</code> statique	<code>api_lan</code> , <code>api_remote</code> , <code>wss_url</code> , VPN, codecs, endpoints	Toujours joignable (CDN GitHub)
API LAN	<code>https://pbx.local/provision</code> (Apache + PHP)	<code>register/verify/claim/session</code> , VPN, groupes, chat	Sur le LAN et via VPN
API distante	Tunnel Cloudflare sortant (URL try-cloudflare)	La même API PHP — le tunnel évite tout port entrant	Republiée au boot dans le bootstrap

i LE DÉTAIL QUI REND LE SYSTÈME ROBUSTE

L'URL trycloudflare change à chaque démarrage (mode quick). Le boot enchaîne donc, dans cet ordre précis : rafraîchir le tunnel → `sync-global-config.sh --deploy` (régénérer le bootstrap avec la nouvelle URL) → publier sur GitHub Pages. Un client hors LAN retrouve ainsi toujours une `api_remote` valide, sans DNS ni domaine dédié.

› Le parcours utilisateur

Côté application, l'utilisateur qui choisit « M'enregistrer » saisit une adresse e-mail — c'est sa seule action « administrative » :

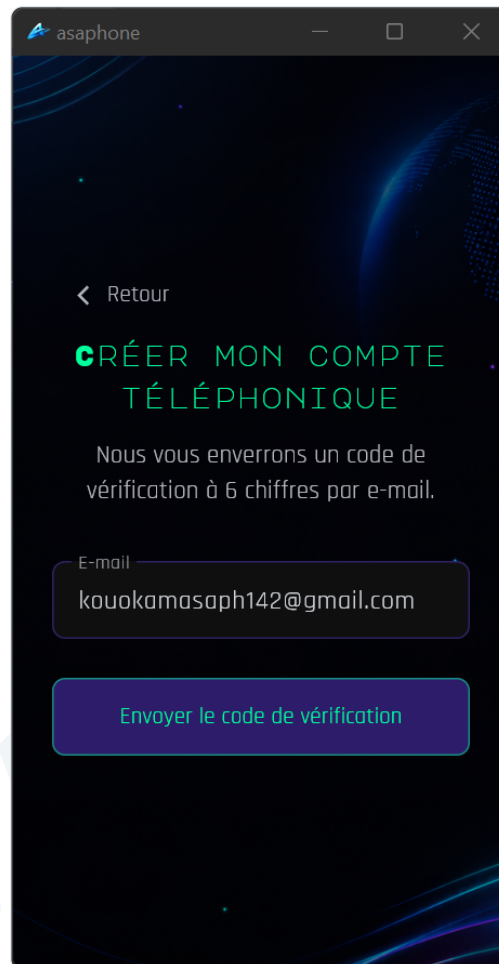


Fig. 18 · écran d'inscription Asaphone — parcours « M'enregistrer », saisie de l'e-mail

Le serveur déroule alors la séquence complète : code à 6 chiffres (TTL 15 minutes, haché en base), vérification, attribution automatique d'une extension libre du pool **1003–1010** (politique `auto`), génération d'un **QR one-shot** (token de claim, validité 24 h) envoyé par e-mail — jamais de secret SIP en clair dans le corps du message. Le scan déclenche le `claim` (credentials complets), le client s'enregistre en WSS, puis révoque son token (`consume, used=1`) : un QR intercepté après coup ne vaut plus rien.

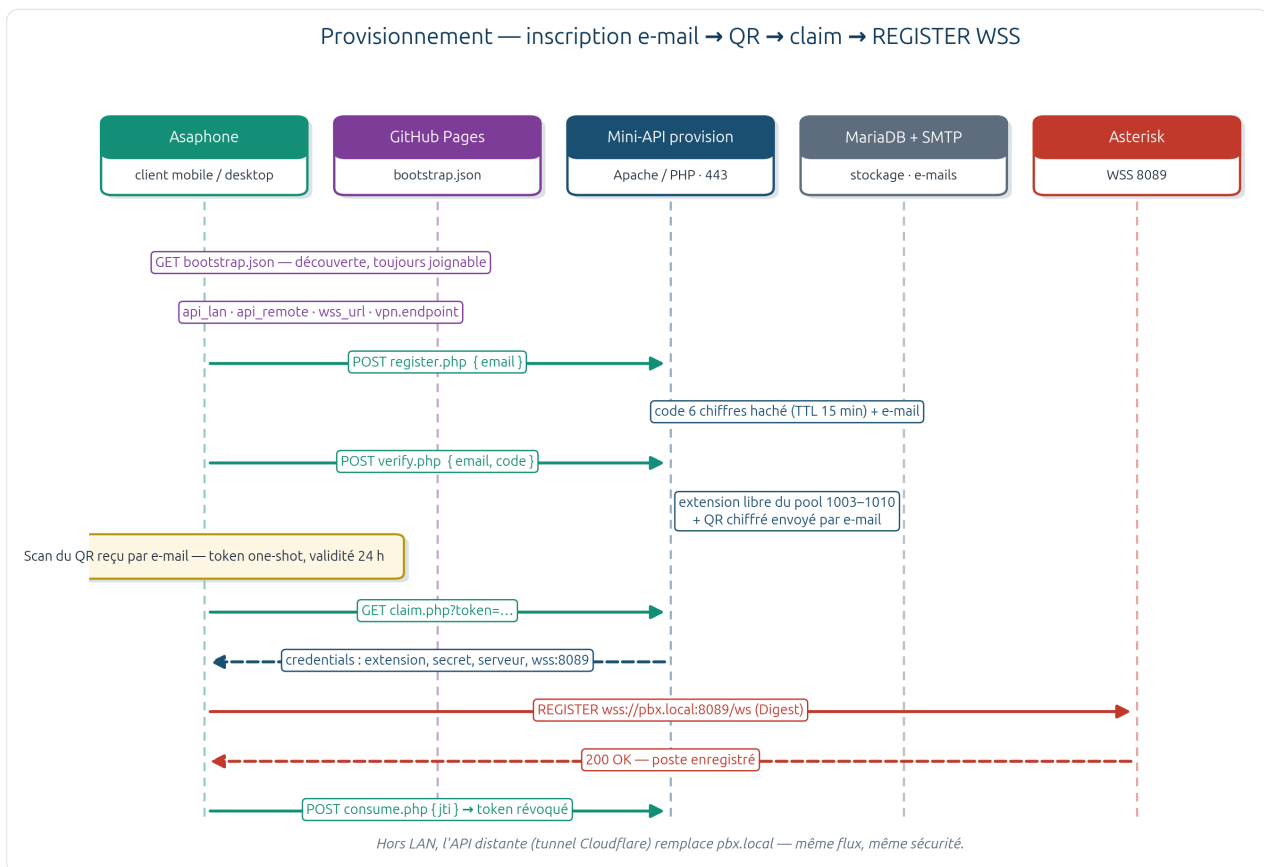


Fig. 19 · séquence de provisionnement — bootstrap → register/verify → QR → claim → REGISTER WSS → consume

› Une réservation honnête du pool

Subtilité de conception qui évite l'épuisement du pool par des inscriptions abandonnées : l'envoi du QR ne *réserve pas* l'extension. Seule l'authentification réussie (REGISTER → consume → statut `provisioned`) marque le poste `taken=true`. L'endpoint `extension.php` expose l'état du pool à tout moment — extension par extension, avec l'e-mail en attente s'il y en a un.

› Défenses du flux

Risque	Contre-mesure
Spam d'inscriptions	Rate-limit par IP <i>et</i> par e-mail (fenêtre glissante 1 h, table dédiée)
Énumération d'adresses	Réponse générique « si l'e-mail existe, un message a été envoyé »
Interception du code	TTL 15 min, code haché (bcrypt), transport HTTPS
Rejeu du QR	Token one-shot : jti révoqué au premier claim/REGISTER, expiration 24 h
Fuite de secret par e-mail	Le QR chiffré est autoporteur — aucun mot de passe en clair dans le mail
Accès à l'admin via l'API	Alias Apache /provision isolé de l'UI FreePBX, sans session admin

◆ L'ESSENTIEL DU CHAPITRE

- Découverte en trois étages : bootstrap GitHub (stable) → API LAN → API tunnel (republiée au boot).
- Onboarding complet en 4 gestes utilisateur : e-mail, code, scan, appel — moins de deux minutes.
- QR one-shot révoqué à l'usage ; le pool ne se réserve qu'à l'authentification réelle.
- Zéro backend cloud : Apache + PHP + MariaDB + SMTP, le tout sur le PBX.

→ ET ENSUITE ?

Le compte existe, le poste est enregistré — sur le LAN. La partie IV s'attaque au reste du monde : accès distant, supervision et exploitation quotidienne.

ASAPHONE
RAPPORT TECHNIQUE · 2026

Exploitation

Joindre le PBX de partout, l'observer en continu, et le laisser se réparer tout seul à chaque démarrage.

13 Accès distant : VPN WireGuard, tunnels et trunks

14 Supervision : Telegraf → InfluxDB → Grafana

15 Installation et démarrage automatisé

16 Validation et tests

ASAPHONE
RAPPORT TECHNIQUE - 2026

13 Accès distant : VPN WireGuard, tunnels et trunks

Cinq mécanismes de liaison qui ne font pas la même chose — et le scénario qui les départage.

› Le scénario révélateur

Le document VPN du dépôt part d'un cas concret : M. Dupont, softphone opérationnel au bureau, part travailler depuis un réseau domestique (192.168.137.0/24). Résultat immédiat — « Registration failed ». Rien n'est en panne : mDNS ne traverse pas Internet, l'IP privée du PBX est injoignable, et quand bien même une route existerait, UFW et les localnets refuseraient cette source inconnue. La leçon : **aucun réglage téléphonique ne résout un problème de couche 3**. Il faut un pont réseau — et c'est précisément ce que chaque mécanisme du tableau suivant fait... ou ne fait pas :

Cinq façons de relier — extension, trunk, VPN, VLAN, tunnel					
	Extension PJSIP	Trunk SIP	VPN WireGuard	VLAN 10 (802.1Q)	Tunnel Cloudflare
Couche	SIP (L7) — poste	SIP (L7) — inter-PBX ou opérateur	IP (L3) — kernel	Ethernet (L2) — switch	HTTPS (L7) — sortant
Relie quoi ?	Poste ↔ PBX (1001–1010)	PBX ↔ PSTN ou autre PBX	PC distant ↔ LAN PBX (10.200.0.0/24)	Téléphones du site ↔ PBX (10.10.10.0/24)	Internet ↔ API provision (443)
Authentification	REGISTER + secret (≥ 16 caractères)	Login opérateur ou identifi IP	Paires de clés WireGuard	Aucune — tag VLAN sur ports switch	Tunnel sortant, aucun port entrant
Où configurer ?	FreePBX → Extensions (scripts phase 2)	FreePBX → Trunks (routes préparées)	OS Linux (wg0) + pare-feu	Switch + hyperviseur + ens33.10	Service systemd dédié
Softphone distant ?	Oui — via VPN ou relais WSS	Non	Oui (télétravail)	Non (site uniquement)	Non (API HTTP seulement)
État projet	production 1001–1010	préconfiguré (secrets à fournir)	opérationnel (enrôlement API)	OS prêt — switch à raccorder	actif au boot (URL éphémère)

Fig. 20 · comparatif — extension PJSIP, trunk SIP, VPN WireGuard, VLAN 802.1Q, tunnel Cloudflare

› La réponse : WireGuard enrôlé par l'API

Le VPN retenu est **WireGuard** (interface `wg0`, PBX en 10.200.0.1, UDP 51820) — et son enrôlement est aussi automatisé que le reste : `POST vpn/enroll.php` avec un `device_id` stable retourne une URL de claim ; `GET vpn/claim.php` délivre la configuration WireGuard complète (avec les indications SIP/WSS). Pas de compte, pas d'e-mail pour ce flux — la révocation d'un appareil se fait par `vpn/revoke.php`. Côté serveur, seule règle à retenir : autoriser le **sous-réseau du tunnel** (10.200.0.0/24) dans `EXTRA_LAN_CIDRS` — jamais le LAN distant de l'utilisateur, que le PBX ne verra jamais.

! LE CAS CGNAT (STARLINK) — PAS D'UDP ENTRANT DU TOUT

Derrière un CGNAT, même le port 51820 est inaccessible. La parade du projet : un **relais WireGuard-sur-WebSocket** (`install-wg-wss-relay.sh`) exposé par tunnel Cloudflare *sortant*. L'application ouvre le relais WSS, puis établit WireGuard vers 127.0.0.1:51820 à travers lui. Le bootstrap publie `tunnel.wss_url` — le client bascule seul, sans app externe.

› Windows et pbx.local

Dernier maillon de l'accès distant, plus prosaïque : Windows ne résout pas toujours les noms mDNS `.local`. Le profil réseau régénère donc à chaque application le fichier `network/windows-hosts.txt`, dont la ligne se copie (en administrateur) dans le fichier `hosts` du poste :

```
# C:\Windows\System32\drivers\etc\hosts
192.168.1.80 pbx.local pbx # exemple généré – réseau 192.168.1.0/24
```

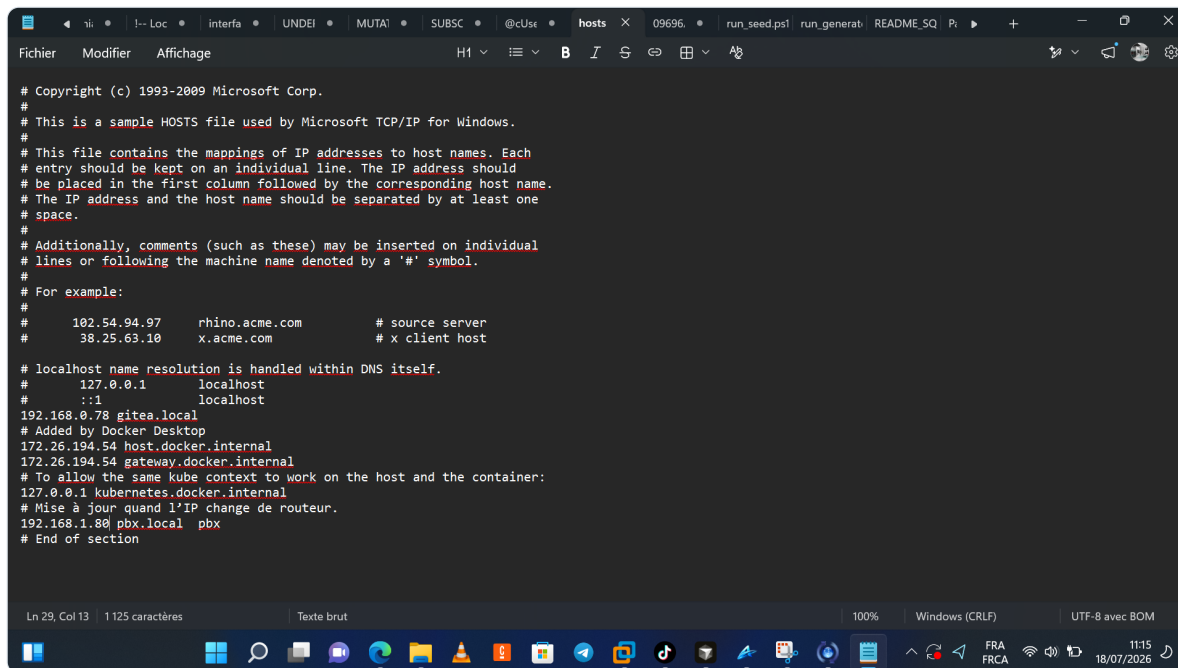


Fig. 21 · édition du fichier `hosts` Windows pour résoudre `pbx.local` (poste sans mDNS)

› Et les trunks ?

Il faut le dire clairement, car la confusion est fréquente : **rien de tout cela n'est un trunk**. Toute la téléphonie interne — appels mixtes UDP↔WSS, conférences, télétravail — fonctionne sans trunk, par extensions PJSIP et B2BUA. Le trunk SIP ne devient nécessaire que pour le PSTN (numéro public, appels sortants) ou pour relier un second PBX. Le dépôt les a préparés : `trunk-operateur-pstn` et `trunk-interpbx-site-b` avec leurs routes (France, international, inter-PBX, entrante catch-all → IVR 7000), activables dès que les identifiants opérateur seront renseignés — hors Git, dans `/root/trunks-secrets.env`.

◆ L'ESSENTIEL DU CHAPITRE

- Un softphone hors site est un problème de couche 3 : la réponse est le VPN, pas un réglage SIP.
- Enrôlement WireGuard sans compte (`device_id` → `claim.conf`) ; révocation par API.
- CGNAT : relais WG-sur-WSS via tunnel Cloudflare sortant — aucune ouverture de port.
- Trunks PSTN/inter-PBX préparés mais distincts : la téléphonie interne n'en a pas besoin.

→ ET ENSUITE ?

L'accès est réglé ; encore faut-il voir ce qui se passe. Le chapitre 14 branche les instruments de mesure sur le cœur du PBX.

14 Supervision : Telegraf → InfluxDB → Grafana

Observer sans peser : trois conteneurs, un utilisateur AMI dédié, des dashboards versionnés.

› Une chaîne courte et assumée

La stack tient en trois conteneurs Docker Compose (`monitoring/`) et un choix assumé : **pas de Prometheus**. Les métriques suivent la chaîne Telegraf → InfluxDB 2 → Grafana (langage Flux) — plus simple à opérer pour une personne seule. Particularité technique : les images Telegraf officielles n'ont pas de plugin Asterisk ; le dépôt construit donc `voip-telegraf:1.33-ami`, qui embarque deux scripts Python appelés toutes les 10 secondes — `ami_metrics.py` (interroge l'AMI : canaux, appels actifs → mesure `asterisk_core`) et `log_metrics.py` (lit les logs Asterisk et Fail2Ban montés en lecture seule). Le conteneur tourne en `network_mode: host` pour joindre l'AMI sur `127.0.0.1:5038` — avec un utilisateur AMI dédié `telegraf`, limité mais doté de `write=command` (nécessaire à `core show channels`).

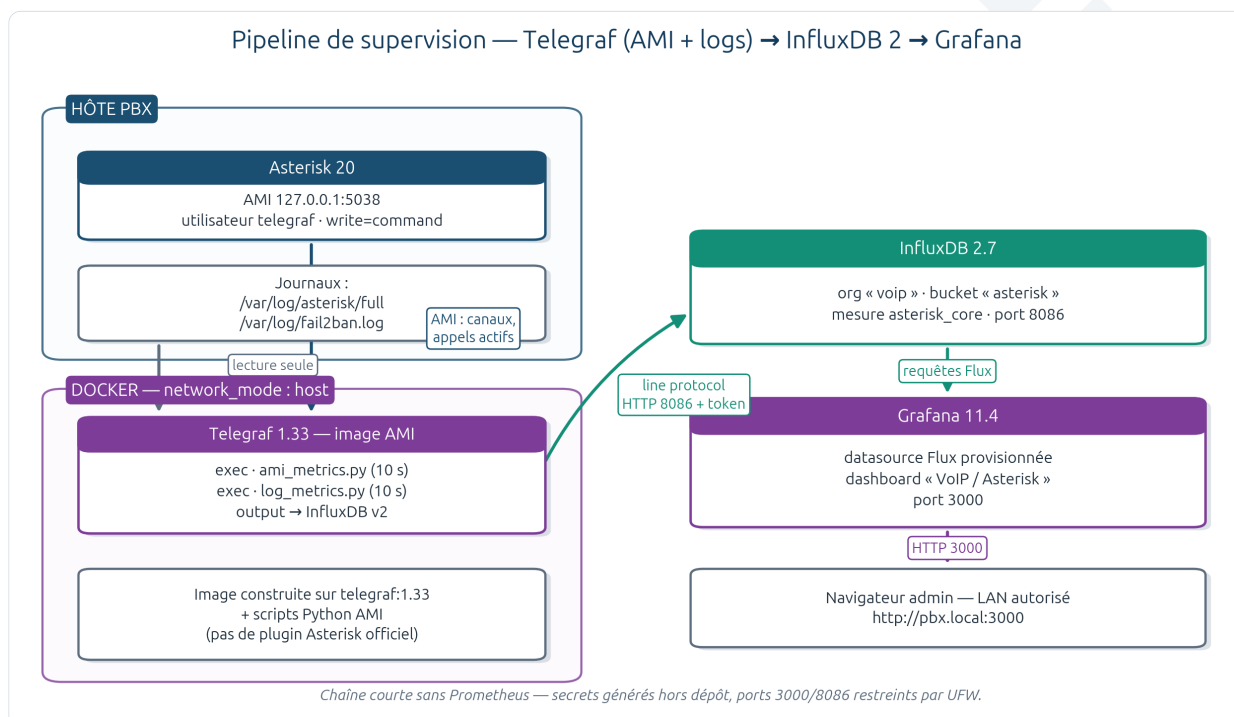


Fig. 22 · pipeline de supervision — Telegraf (AMI + logs) → InfluxDB 2 → Grafana

› En console

```

Supervision — docker compose ps / logs

$ cd /home/asaph/Documents/serveur/monitoring && docker compose ps
NAME                IMAGE                                STATUS    PORTS
voip-influxdb       influxdb:2.7-alpine                Up (healthy)    0.0.0.0:8086->8086/tcp
voip-grafana        grafana/grafana:11.4.0             Up            0.0.0.0:3000->3000/tcp
voip-telegraf       voip-telegraf:1.33-ami             Up            network_mode: host

$ docker compose logs --tail 6 telegraf
voip-telegraf | I! Starting Telegraf 1.33.0
voip-telegraf | I! Loaded inputs: exec (2x: ami_metrics.py, log_metrics.py)
voip-telegraf | I! Loaded outputs: influxdb_v2
voip-telegraf | I! [agent] interval:10s, flush_interval:10s, host "freepbx"
voip-telegraf | D! ami_metrics: AMI login 127.0.0.1:5038 user=telegraf OK
voip-telegraf | D! asterisk_core channels=2i, calls_active=1i → bucket asterisk

Rappel ports : Grafana → http://pbx.local:3000 (dossier VoIP)
InfluxDB 2 → http://pbx.local:8086 (org voip, bucket asterisk)
UFW : 3000/tcp et 8086/tcp ouverts uniquement depuis les LAN autorisés.
  
```

Fig. 23 · état de la stack et logs Telegraf — docker compose ps / logs

› Dans le navigateur

Grafana est *provisionné par fichiers* — datasource InfluxDB-VoIP et dashboard « VoIP / Asterisk — monitoring » sont versionnés dans le dépôt (`grafana/provisioning/` , `grafana/dashboards/voip-asterisk.json`) et chargés au démarrage du conteneur : un `docker compose up -d` sur une machine vierge reconstruit l'observabilité à l'identique.

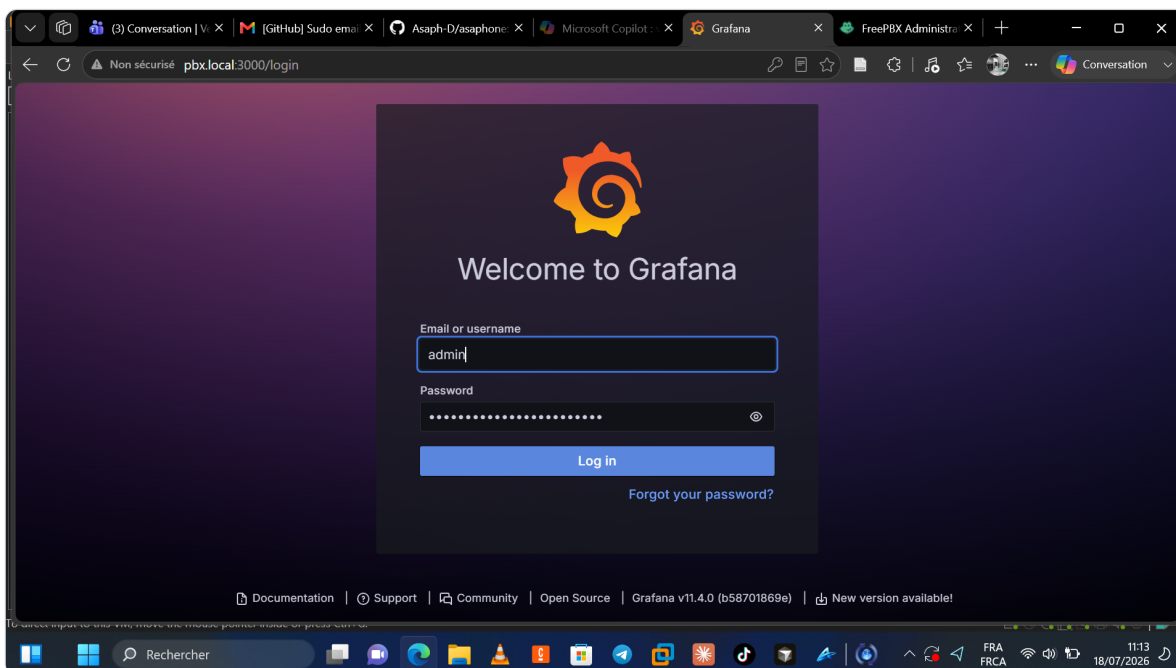


Fig. 24 · page d'accueil Grafana (port 3000) après démarrage de la stack

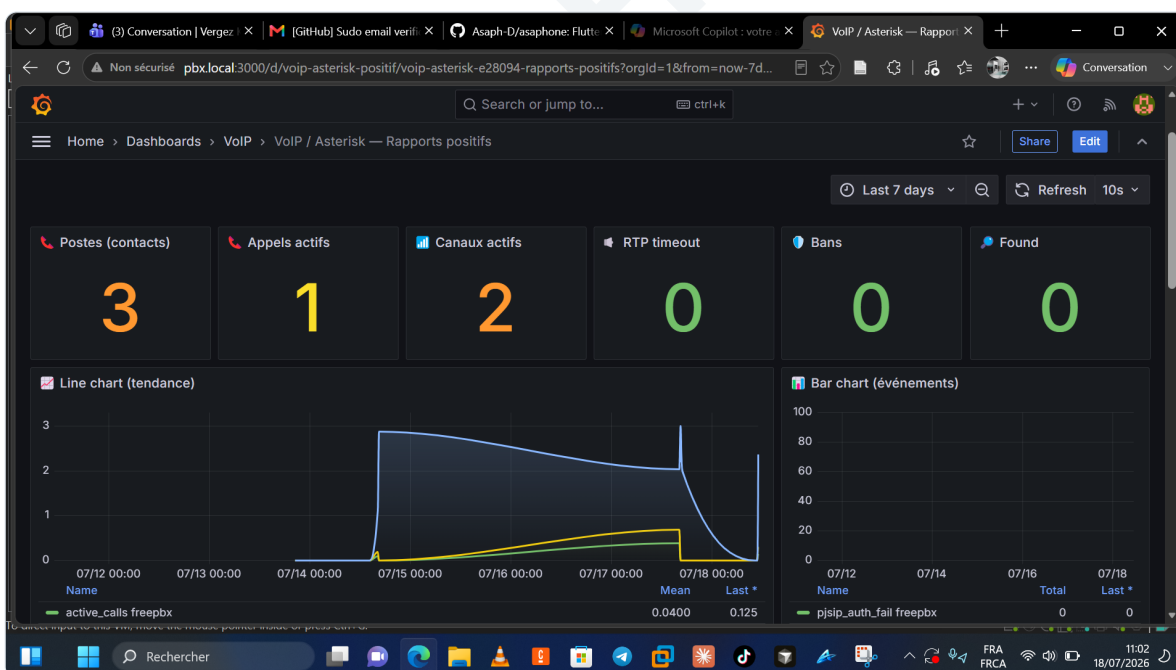


Fig. 25 · dashboard « VoIP / Asterisk — monitoring » — canaux et appels actifs

◆ L'ESSENTIEL DU CHAPITRE

- Trois conteneurs, zéro Prometheus : Telegraf (scripts AMI Python) → InfluxDB 2 → Grafana Flux.
- Image Telegraf maison en network host ; utilisateur AMI dédié, secrets générés dans `monitoring/.env`.
- Dashboards et datasource versionnés : l'observabilité se reconstruit d'un `docker compose up`.
- Ports 3000/8086 restreints aux LAN autorisés par UFW.

→ ET ENSUITE ?

Dernière pièce de l'exploitation, et non la moindre : comment tout cela s'installe, et surtout comment tout cela redémarre — seul. Chapitre 15.

15 Installation et démarrage automatisé

D'un OS vierge à un PBX complet — puis un boot en 17 étapes qui se répare lui-même.

› Installer : quatre jalons

Le déploiement sur machine vierge (Debian 12 / Ubuntu 22.04) suit INSTALLATION.md en quatre jalons, chacun doté d'un critère de réussite vérifiable — les planches ci-dessous les condensent, versions strictement issues du dépôt :

```

Installation 1/4 — prérequis système

ÉTAPE 1/6 — Prérequis OS et paquets système
Cible : Debian 12 (Bookworm) ou Ubuntu 22.04 LTS — accès root + systemd

$ sudo apt-get update
$ sudo apt-get install -y \
    apache2 mariadb-server \
    ufw avahi-daemon \
    fail2ban \
    libsrtp2-dev \
    python3 \
    git curl

$ apache2 -v
Server version: Apache/2.4.x (Ubuntu)
$ mariadb --version
mariadb ... 10.6+ (Debian 12 : souvent 10.11)
$ php -v
PHP 8.2.x (cli)

✓ Critère OK : apache2, mariadb, php, ufw, avahi, fail2ban installés

```

Fig. 26 · installation 1/4 — prérequis OS et paquets système

```

Installation 2/4 — Asterisk 20 + FreePBX 17

ÉTAPE 2/6 — Asterisk 20 LTS · ÉTAPE 3/6 — FreePBX 17
Installation via le guide officiel FreePBX (script Sangoma ou ISO Distro).
Référence projet : Asterisk 20.18.2 (sources /usr/src/asterisk-20.18.2)

$ asterisk -V
Asterisk 20.18.2

$ fwconsole -V
FreePBX 17.0.x

Module SRTP (WebRTC) — si compilation source sans libsrtp2 :
$ sudo bash serveur/scripts/install-asterisk-res-srtp.sh
$ asterisk -rx "module show like res_srtp"
res_srtp.so    Secure RTP (SRTP)    ... Running

✓ Critère OK : asterisk -V → 20.x · fwconsole -V → 17.x · res_srtp chargé

```

Fig. 27 · installation 2/4 — Asterisk 20 LTS et FreePBX 17 (contrôles asterisk -V, fwconsole -V, res_srtp)

```

Installation 3/4 — Docker + stack monitoring

ÉTAPE 4/6 — Docker + Compose v2 · ÉTAPE 5/6 — Stack monitoring

$ sudo apt-get install -y docker.io docker-compose-plugin
$ docker --version
Docker version 24+ ...
$ docker compose version
Docker Compose version v2.x

$ cd serveur/monitoring
$ bash ../scripts/phase3-gen-monitoring-env.sh # génère .env (secrets)
$ docker compose pull
influxdb Pulled influxdb:2.7-alpine
grafana Pulled grafana/grafana:11.4.0
telegraf Pulled telegraf:1.33 (base du build)
$ docker compose build
=> naming to docker.io/library/voip-telegraf:1.33-ami done
$ docker compose up -d
[+] Running 3/3
✓ Container voip-influxdb Started
✓ Container voip-grafana Started
✓ Container voip-telegraf Started

✓ Critère OK : Grafana http://pbx.local:3000 · InfluxDB http://pbx.local:8086

```

Fig. 28 · installation 3/4 — Docker Compose v2 et stack monitoring

```

Installation 4/4 — service systemd serveur-startup

ÉTAPE 6/6 — Service systemd serveur-startup.service
Adapter d'abord ExecStart= au chemin réel du dépôt (INSTALLATION.md §2).

$ sudo install -m 0644 serveur/systemd/serveur-startup.service \
/etc/systemd/system/serveur-startup.service
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now serveur-startup.service
Created symlink /etc/systemd/system/multi-user.target.wants/
serveur-startup.service → /etc/systemd/system/serveur-startup.service.

$ systemctl status serveur-startup.service --no-pager -l
● serveur-startup.service - Démarrage PBX (FreePBX/Asterisk + réseau site)
   Loaded: loaded (/etc/systemd/system/serveur-startup.service; enabled)
   Active: active (exited) – journal : /var/log/serveur-startup.log

Optionnel documenté : WireGuard (docs/implement-VPN.md), tunnel Cloudflare
(install-provision-tunnel.sh), Apache HTTPS (enable-apache-https.sh).

✓ Critère OK : service enabled · log /var/log/serveur-startup.log alimenté

```

Fig. 29 · installation 4/4 — service systemd serveur-startup (ExecStart à adapter au chemin réel)

› Le boot : un contrat en 17 étapes

Le principe du chapitre 3 — « le boot est le contrat » — se matérialise dans `server-startup.sh`. Sa propriété clé est la **résilience hors ligne** : les étapes sont classées *critiques* (Apache, profil réseau — un échec marque WARN) ou *optionnelles* (tout ce qui touche Internet — un échec marque SKIP et le boot continue). Un serveur qui démarre dans une cave sans réseau offre quand même la téléphonie interne complète. Les trois planches suivantes reproduisent fidèlement le déroulé et le style console du script (banner, progression, ✓/△/○) :

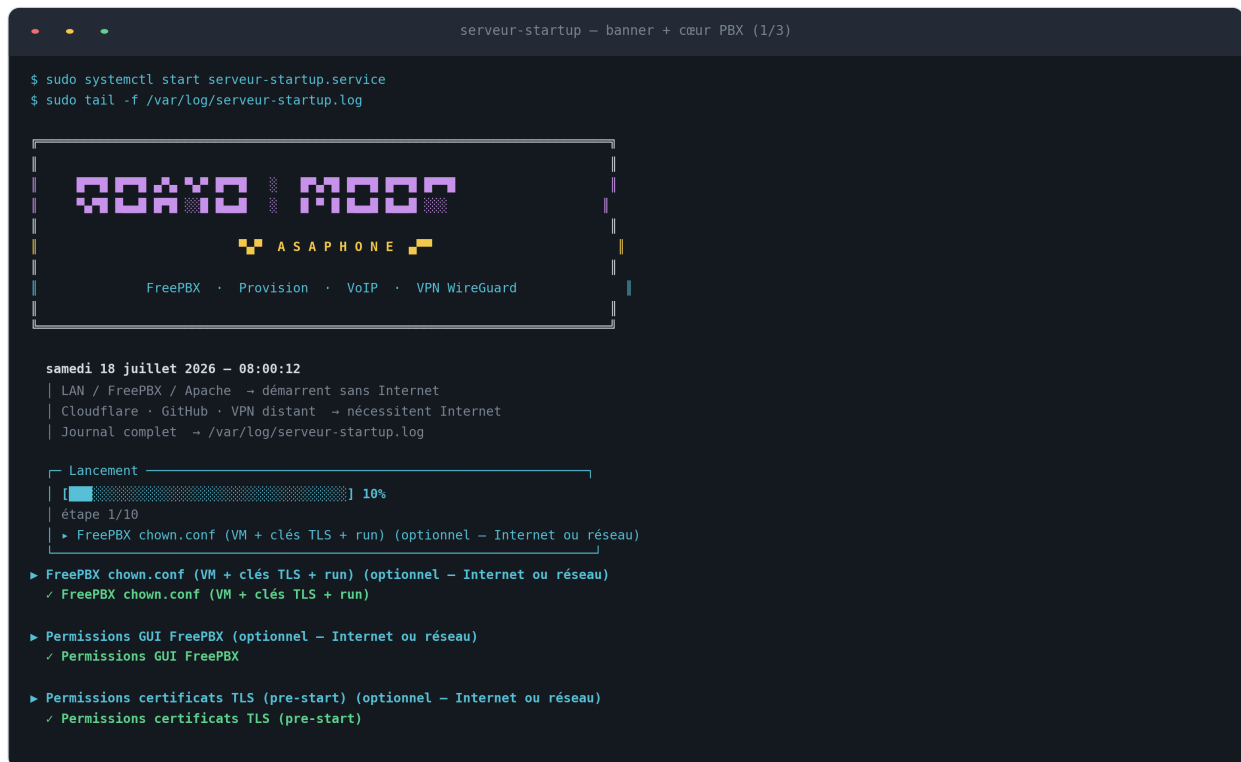


Fig. 30 · serveur-startup 1/3 — banner ASAPHONE, rappels hors-ligne, premières étapes FreePBX

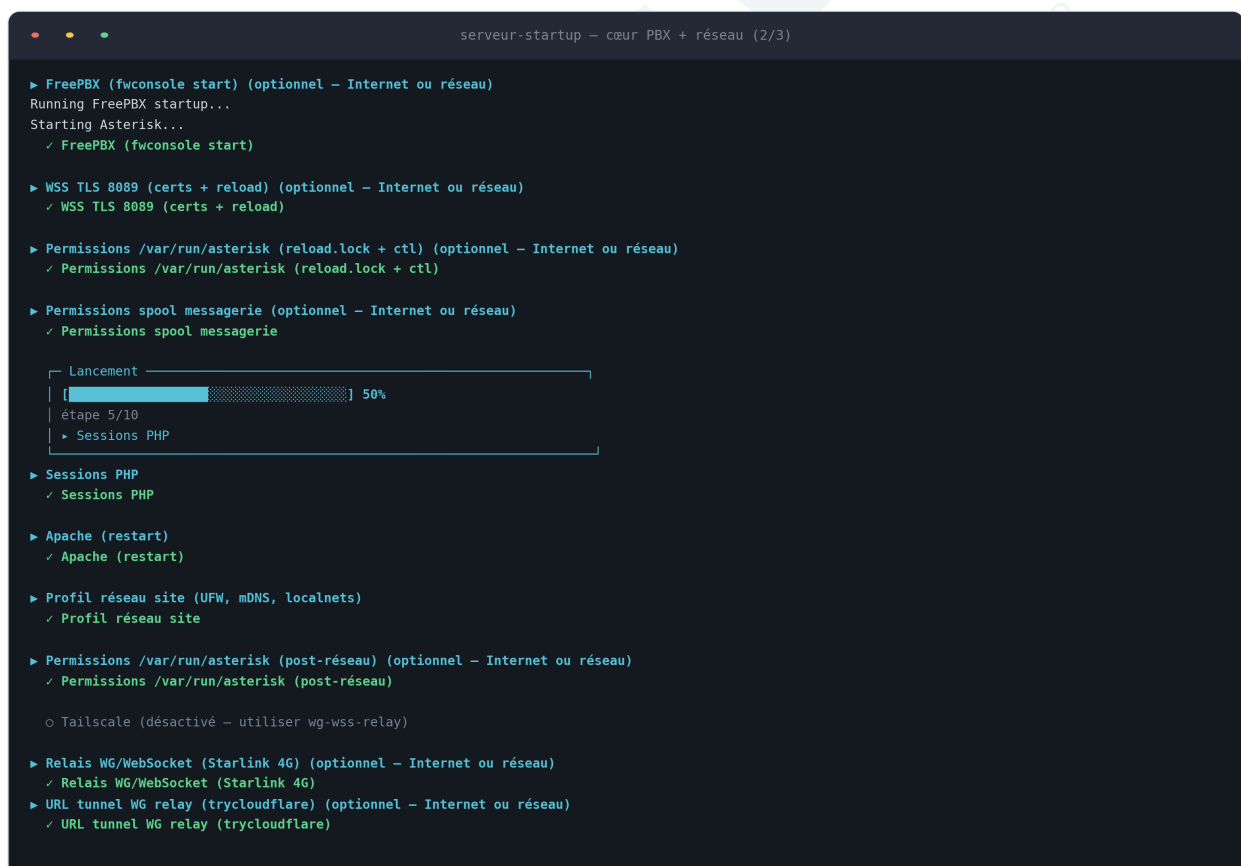


Fig. 31 · serveur-startup 2/3 — fwconsole, WSS 8089, permissions, Apache, profil réseau, relais WG

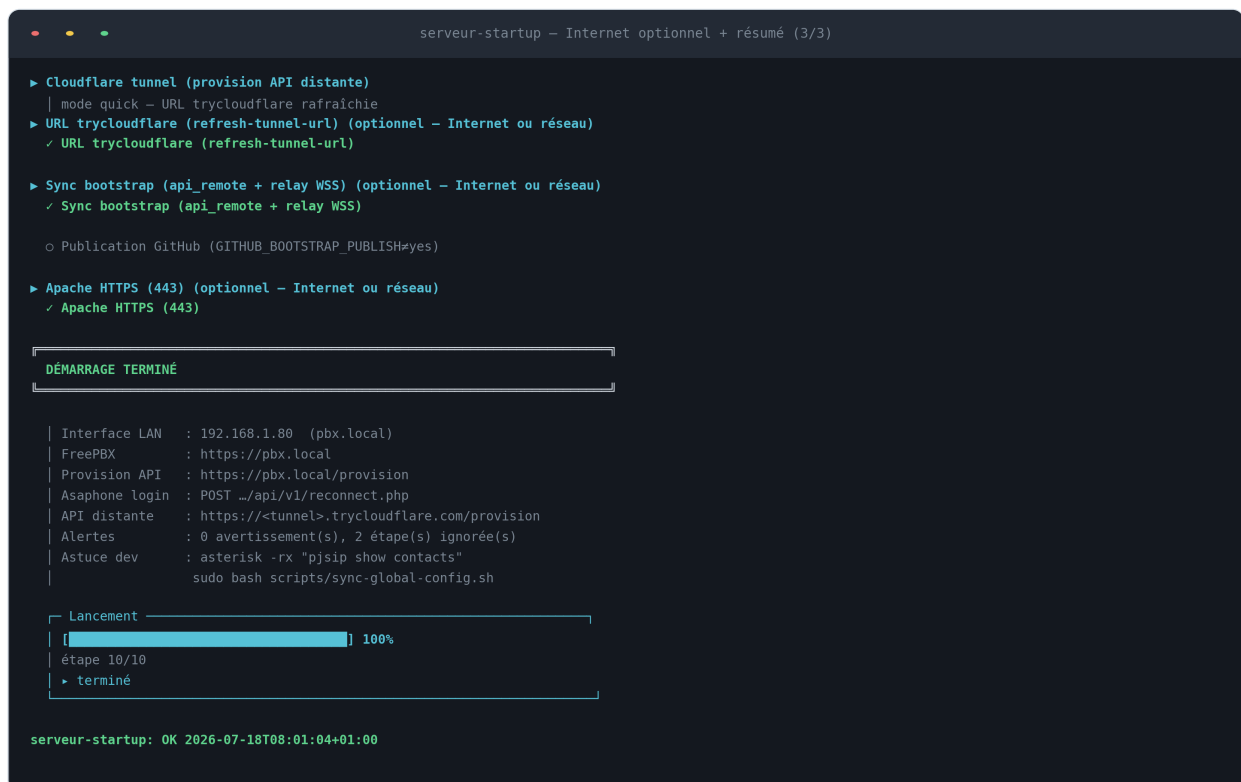


Fig. 32 · serveur-startup 3/3 — Internet optionnel (tunnel, sync bootstrap), HTTPS 443 et résumé final

► Pourquoi cet ordre-là

L'ordre des étapes n'est pas arbitraire — il encode les leçons d'exploitation du projet : les **permissions avant fw-console** (sinon le chown automatique de FreePBX casse les clés TLS), le **WSS juste après le démarrage** (sinon le port 8089 reste fermé), le **réseau après le cœur** (UFW et localnets supposent Apache et Asterisk debout), et surtout la **synchronisation du bootstrap après le rafraîchissement des tunnels** — faute de quoi GitHub publierait une API distante périmée.

Bloc	Étapes	Comportement en échec
Cœur PBX (sans Internet)	chown.conf → perms GUI → certs TLS → fwconsole start → WSS 8089 → perms run/spool	SKIP par étape, le boot continue
Système local	Sessions PHP → Apache restart → profil réseau site → perms post-réseau	Critique : WARN + indicateur hors-ligne
Internet optionnel	Relais WG/WSS → tunnel Cloudflare → sync bootstrap --deploy → publication GitHub → HTTPS 443	SKIP silencieux (retenté au prochain boot ou à la main)
Bilan	Résumé (IP LAN, pbx.local, API distante, alertes) + serveur - startup: OK <date>	—

► Au quotidien : une seule règle

Après toute modification dans l'UI FreePBX, un geste — et un seul — matérialise le changement : **Apply Config** (`fwconsole reload`), qui régénère les fichiers depuis la base :

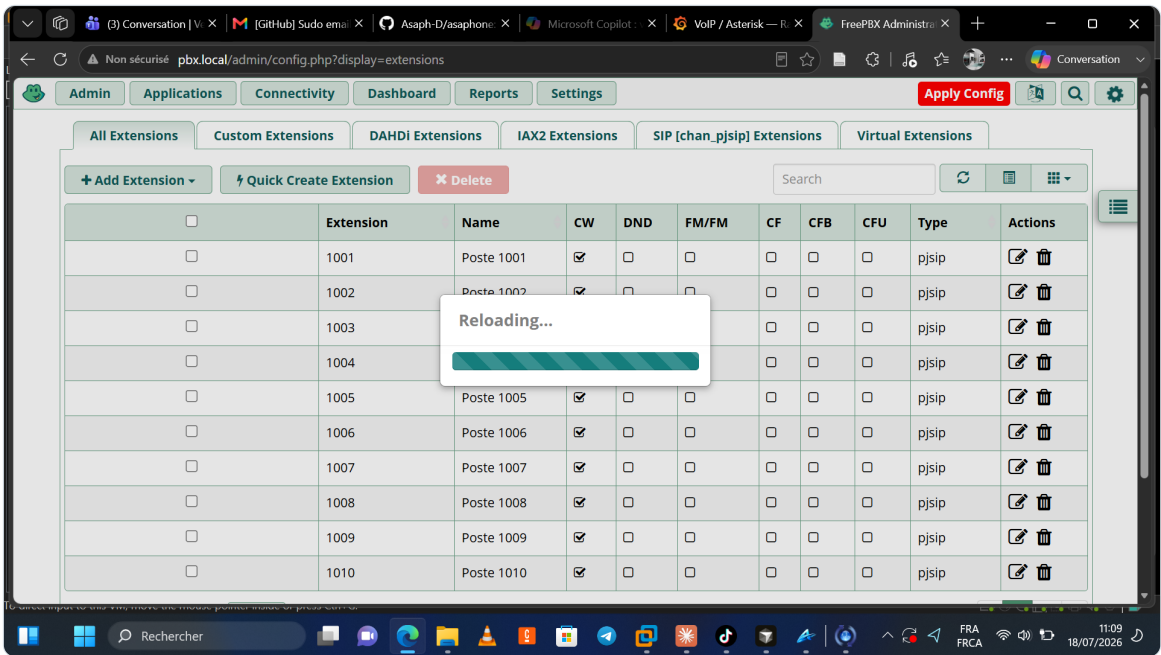


Fig. 33 · application des changements FreePBX — Apply Config (fwconsole reload)

◆ L'ESSENTIEL DU CHAPITRE

- Installation en 4 jalons vérifiables ; versions épinglées par le dépôt (Asterisk 20.18.2, FreePBX 17, Influx 2.7, Grafana 11.4, Telegraf 1.33).
- Boot en 17 étapes : critiques vs optionnelles — la téléphonie interne survit à l'absence d'Internet.
- L'ordre encode les leçons : permissions → fwconsole → WSS → réseau → tunnels → bootstrap → publication.
- Un seul geste quotidien : Apply Config après chaque modification d'UI.

→ ET ENSUITE ?

Reste à prouver que tout cela fonctionne. Le chapitre 16 rassemble les validations — celles déjà consignées dans le dépôt et les commandes pour les rejouer.

16

Validation et tests

La modalité « production d'abord » en actes : chaque brique a son contrôle, rejouable en une commande.

Domaine	Validation consignée	Comment la rejouer
Appels mixtes	1001 (UDP) → 1003 (WSS) : sonnerie, décrochage, bridge établi	Appel réel + <code>pjsip set logger on</code>
QoS	Règles mangle EF présentes ; contrôle en appel réel prescrit	<code>iptables -t mangle -L OUTPUT -n -v · tcpdump portrange 10000-20000</code>
Extensions	10 endpoints PJSIP + mapping voicemail en base	<code>pjsip show endpoints · SELECT sur users</code>
Messagerie	default 1001...1010 listées ; dépôt après bip validé	<code>voicemail show users · appel test</code>
WSS/TLS	HTTPS 8089 actif, transport wss 0.0.0.0:8089	<code>http show status · pjsip show trans-ports · ss -tlnp</code>

Domaine	Validation consignée	Comment la rejouer
Fail2Ban	Filtre validé sur extrait de log réel	<code>phase4-test-fail2ban-filter.sh 2000</code>
Provision	Scénario complet register → verify → claim → consume en curl	Commandes du flux d'onboarding (§15 du document source)
Monitoring	Mesure asterisk_core visible dans Grafana	Explore → Flux <code>from(bucket:"asterisk")</code>
Boot	Journal complet du démarrage + résumé OK	<code>journalctl -u serveur-startup.service -b</code>

› Le réflexe santé en sept commandes

```
$ sudo asterisk -rx "pjsip show contacts" # qui est enregistré ?
$ sudo asterisk -rx "http show status" # WSS 8089 actif ?
$ ss -tlnp | grep -E '8089|443|3000' # les ports écoutent ?
$ sudo ufw status numbered # le pare-feu est conforme ?
$ sudo fail2ban-client status asterisk # qui est banni ?
$ curl -sk https://pbx.local/provision/ # l'API répond ?
$ tail -50 /var/log/serveur-startup.log # le dernier boot ?
```

⚠ POINT DE VIGILANCE OUVERT

L'avertissement « DTLS packet dropped. ICE not completed yet » apparaît en début d'appel WebRTC. Diagnostic documenté : négociation ICE côté client, sans impact constaté sur l'établissement — mais c'est la première piste si l'audio devient instable (vérifier ICE/STUN et l'ouverture RTP).

◆ L'ESSENTIEL DU CHAPITRE

- Chaque domaine a une validation consignée et une commande de rejou — le « terminé » est vérifiable.
- Le flux de provisionnement se teste intégralement en curl, sans client mobile.
- Un point ICE/DTLS côté client est tracé comme vigilance ouverte, sans impact fonctionnel constaté.

→ ET ENSUITE ?

Le système fonctionne et se vérifie. La partie V prend du recul : ce qui manque encore, et ce que le projet démontre.

Bilan

Un regard honnête sur les limites, les chantiers ouverts — et ce que cette plateforme prouve déjà.

17 Limites et perspectives

18 Conclusion

ASAPHONE
RAPPORT TECHNIQUE · 2026

17 Limites et perspectives

Aucune n'est cachée : chaque limite est tracée dans un document du dépôt, avec sa parade prévue.

› Ce qui manque encore — et pourquoi ce n'est pas bloquant

Limite	Impact réel	Parade prévue
Trunk VLAN 10 physique non raccordé (switch/hyperviseur)	Les téléphones du VLAN voix ne sont pas encore en production ; le LAN gestion porte les postes actuels	Config OS déjà prête : trunk 802.1Q + port group, puis trust DSCP
Trunks opérateur inactifs (secrets non renseignés)	Pas d'appels PSTN ni de numéro public	apply-trunks.sh + /root/trunks-secrets.env ; routes déjà écrites
SMTP par poste non configuré	Notification e-mail de la messagerie vocale inopérante (la notification Asaphone, elle, fonctionne)	Adresse + SMTP par extension dans l'UI
URL trycloudflare éphémères (mode quick)	Dépendance à la republication du bootstrap à chaque boot	Tunnel nommé + domaine propre (CLOUDFLARE_TUNNEL_MODE=named)
Secrets SIP réversibles en base	Limitation structurelle FreePBX	Accès base restreint + rotation 90 j ; documenté plutôt que contourné
Pas d'E2EE strict poste-à-poste	Le PBX voit le média (nécessaire aux services)	Modèle de menace assumé : chiffrement par segment
Module GUI ringroups absent	Groupes gérés par dialplan custom (8000)	Réinstallable quand le miroir Sangoma répond
NFS enregistrements non monté	Les enregistrements restent sur le disque local	Export NFS vers /var/spool/asterisk/monitor côté infra
MFA admin / chiffrement disque	Non déployés	Roadmap sécurité (module ou reverse proxy)

› Les cinq chantiers suivants, dans l'ordre

- **1. Raccorder le VLAN 10** — dernier maillon entre la conception réseau et sa réalité physique.
- **2. Activer un trunk opérateur** (TLS si l'offre le permet) — l'IVR 7000 attend déjà les appels entrants.
- **3. Passer au tunnel Cloudflare nommé** avec domaine dédié — bootstrap stable, certificat Let's Encrypt sur l'UI.
- **4. Automatiser la rotation des secrets SIP** (90 j) — aujourd'hui procédure manuelle documentée.
- **5. Enrichir la supervision** — files 7020, bannissements Fail2Ban et qualité RTP à partir des métriques log déjà collectées.

◆ L'ESSENTIEL DU CHAPITRE

- Neuf limites tracées, aucune bloquante pour l'usage interne actuel.
- Les parades sont déjà préparées par la configuration en place (trunks écrits, mode named prévu, NFS documenté).
- Priorité 1 : le raccordement physique du VLAN — tout le reste du socle voix l'attend.

18 Conclusion

Retour à la scène d'ouverture : la simplicité des deux minutes était le produit de tout le reste.

La collaboratrice de l'entrée de jeu ne le saura jamais, mais ses deux minutes d'onboarding ont traversé l'intégralité de ce rapport : un **bootstrap** publié au boot par un service systemd résilient, une **mini-API** isolée derrière Apache avec rate-limit et tokens one-shot, un **pool d'extensions** créé par scripts idempotents, un **transport WSS** dont les certificats et permissions sont réappliqués à chaque démarrage, un **pare-feu** qui ne connaît que des réseaux nommés, et un **VLAN** prêt à prioriser chacun de ses paquets de voix.

Sur les quatre tensions de la problématique, le bilan est net. **Qualité contre mutualisation** : résolue par conception (VLAN 10 + DSCP EF), en attente de son raccordement physique. **Ouverture contre surface d'attaque** : résolue par le trio deny-par-défaut / Fail2Ban / accès distant exclusivement tunnelé — au point qu'aucun port entrant n'est requis pour l'API distante. **Simplicité contre rigueur** : résolue par le provisionnement QR one-shot, qui distribue des secrets de 16 caractères sans que personne ne les voie jamais. **Automatisation contre fragilité** : résolue par le boot à deux vitesses, où l'Internet est un bonus et non une condition.

Au-delà de la téléphonie, le projet démontre une méthode : **une infrastructure racontable**. Chaque choix est un document, chaque document un script, chaque script un contrôle. C'est cette chaîne — plus encore que les 10 extensions ou les 17 étapes de boot — qui permet à une personne seule d'opérer, de réparer et de faire évoluer une plateforme que l'on croirait exiger une équipe.

◆ EN UNE PHRASE

ASAPHONE prouve qu'une petite structure peut posséder sa téléphonie — chiffrée, supervisée, auto-réparante et agréable à utiliser — sans cloud, sans équipe dédiée, et sans qu'aucun utilisateur n'ait jamais à savoir ce qu'est un co-dec.

A Annexe A — Matrice des ports

Tous les flux réseau de la plateforme, avec leur exposition pare-feu.

Port(s)	Proto	Service	Exposition (UFW)
5060 · 5160	UDP/TCP	SIP — signalisation classique	VLAN voix (+ LAN gestion si autorisé)
5061 · 5161	TCP/TLS	SIP TLS	LAN gestion + VLAN voix — jamais « Anywhere »
10000 – 20000	UDP	RTP / SRTP (média) — marqué DSCP EF	CIDR voix + LAN autorisés
8088	TCP	HTTP Asterisk (WebRTC lab)	LAN autorisés (option)
8089	TCP/TLS	WSS WebRTC — wss://pbx.local:8089/ws	LAN autorisés + VPN
80 · 443	TCP	Apache : UI FreePBX + /provision	LAN / VPN (443 requis pour l'API)
3000	TCP	Grafana	LAN autorisés (option monitoring)
8086	TCP	InfluxDB 2	LAN autorisés (option monitoring)
5038	TCP	AMI Asterisk (collecte Telegraf)	127.0.0.1 uniquement
51820	UDP	WireGuard wg0	Internet (forward box) — ou relais WSS si CGNAT

B Annexe B — Plan de numérotation complet

Extensions, services et codes — la carte mémoire de la plateforme.

Numéro / code	Type	Description
1001 – 1002	Extension classique	SIP UDP/TLS + SRTP SDES (softphones, téléphones IP)
1003 – 1010	Extension WebRTC	WSS + DTLS-SRTP (Asaphone) — pool de provisionnement automatique
8000	Sonnerie générale	Dial simultané des 10 postes, 45 s (dialplan custom)
7000	Accueil horaire	Ouvré → 7010 ; fermé → message
7010	IVR intelligent	AGI Python — VIP, horaires, saisie d'extension
7020	File support	ACD leastrecent, 1001–1010, appels enregistrés
7101 – 7110	Files individuelles	Une file par poste
8001	Conférence à PIN	Salle ConfBridge phase3-<PIN>
8100	Test MixMonitor	Enregistrement + renvoi messagerie 1001
6000	Salle de groupe par défaut	Annoncée par bootstrap (conference.default_call_uri)
6001 – 6099	Salles numériques	Réservées

Numéro / code	Type	Description
asaphone-grp-*	Salles de groupe	Une par groupe Asaphone synchronisé
*81001 – *81010	Messagerie directe	Consultation de la boîte du poste correspondant

C Annexe C — Glossaire

Les termes qui reviennent, définis dans le contexte du projet.

Terme	Définition
AGI / AMI	Interfaces Asterisk : Gateway Interface (scripts appelés par le dialplan) / Manager Interface (contrôle et événements, port 5038)
B2BUA	Back-to-back user agent — le PBX termine chaque jambe d'appel et les pontes ; le média ne circule jamais en direct entre postes
Bootstrap	Fichier JSON statique de découverte (GitHub Pages) : indique au client API, WSS, VPN, codecs — republié à chaque boot
ConfBridge	Application de conférence d'Asterisk : mixage audio côté serveur
DSCP EF	Classe QoS « Expedited Forwarding » (46 / 0x2e) posée sur les paquets RTP pour la file prioritaire
DTLS-SRTP	Chiffrement média WebRTC : clés négociées en DTLS, flux transporté en SRTP
ICE / AVPF / rtcp-mux	Négociation de chemins réseau / profil RTP à retour rapide / RTP+RTCP sur un même port — le triptyque exigé par WebRTC
jti	Identifiant unique de token — support de la révocation one-shot (QR) et de l'authentification des API (X-Provision-Jti)
localnets	Réseaux déclarés « locaux » à PJSIP : pas de réécriture NAT pour ces sources
mDNS	Multicast DNS (Avahi) — résolution de pbx.local sans serveur DNS ; suppléé par le fichier hosts sous Windows
PJSIP	Pile SIP moderne d'Asterisk (chan_pjsip), remplaçante de chan_sip
SDES	Échange des clés SRTP dans le SDP (media_encryption=sdes) — chiffrement média des postes classiques
Trunk SIP	Lien SIP permanent PBX ↔ opérateur ou PBX ↔ PBX — à ne pas confondre avec le trunk VLAN 802.1Q
WSS	WebSocket sécurisé (TLS) — canal de signalisation SIP des clients WebRTC, port 8089

D Annexe D — Table des figures

Chaque illustration du document, son titre et le chapitre où la retrouver.

N°	Titre de la figure	Chapitre
Fig. 1	Architecture globale — Internet → pare-feu → Asterisk/FreePBX → LAN gestion + VLAN voix	Ch. 4

N°	Titre de la figure	Chapitre
Fig. 2	Dual-homing du PBX — patte gestion (MGMT), patte voix (VLAN 10) et tunnel WireGuard, avec les clients de chaque monde	Ch. 4
Fig. 3	Matrice des couches de sécurité L0 → L7 et leur état	Ch. 6
Fig. 4	Tableau de bord FreePBX 17 — l'UI d'administration du PBX	Ch. 7
Fig. 5	Liste des extensions PJSIP dans FreePBX (Applications → Extensions)	Ch. 7
Fig. 6	Création d'une nouvelle extension PJSIP	Ch. 7
Fig. 7	L'extension créée, prête à être rechargée	Ch. 7
Fig. 8	IVR et files d'attente couvrant toutes les extensions (phase 3)	Ch. 8
Fig. 9	Appel de groupe — synchronisation, ConfBridge et originate des membres	Ch. 8
Fig. 10	Appel 1001 (UDP/SRTP) ↔ 1003 (WSS/DTLS-SRTP) — deux jambes négociées séparément, pont sur le PBX	Ch. 9
Fig. 11	Configuration SIP du client Asaphone — serveur, extension, transport WSS	Ch. 9
Fig. 12	Cas d'utilisation Asaphone ↔ PBX — provision, appels, chat, messagerie, groupes, VPN	Ch. 10
Fig. 13	Écran d'accueil / connexion d'Asaphone	Ch. 10
Fig. 14	Page de configuration du client	Ch. 10
Fig. 15	Clavier d'appel (numérotation interne)	Ch. 10
Fig. 16	Appel audio en cours (démonstration)	Ch. 10
Fig. 17	Appel vidéo en cours (démonstration)	Ch. 10
Fig. 18	Écran d'inscription Asaphone — parcours « M'enregistrer », saisie de l'e-mail	Ch. 12
Fig. 19	Séquence de provisionnement — bootstrap → register/verify → QR → claim → REGISTER WSS → consume	Ch. 12
Fig. 20	Comparatif — extension PJSIP, trunk SIP, VPN WireGuard, VLAN 802.1Q, tunnel Cloudflare	Ch. 13
Fig. 21	Édition du fichier hosts Windows pour résoudre pbx.local (poste sans mDNS)	Ch. 13
Fig. 22	Pipeline de supervision — Telegraf (AMI + logs) → InfluxDB 2 → Grafana	Ch. 14
Fig. 23	État de la stack et logs Telegraf — docker compose ps / logs	Ch. 14
Fig. 24	Page d'accueil Grafana (port 3000) après démarrage de la stack	Ch. 14
Fig. 25	Dashboard « VoIP / Asterisk — monitoring » — canaux et appels actifs	Ch. 14
Fig. 26	Installation 1/4 — prérequis OS et paquets système	Ch. 15
Fig. 27	Installation 2/4 — Asterisk 20 LTS et FreePBX 17 (contrôles asterisk -V, fwconsole -V, res_srtp)	Ch. 15
Fig. 28	Installation 3/4 — Docker Compose v2 et stack monitoring	Ch. 15

N°	Titre de la figure	Chapitre
Fig. 29	Installation 4/4 — service systemd serveur-startup (ExecStart à adapter au chemin réel)	Ch. 15
Fig. 30	Serveur-startup 1/3 — banner ASAPHONE, rappels hors-ligne, premières étapes FreePBX	Ch. 15
Fig. 31	Serveur-startup 2/3 — fwconsole, WSS 8089, permissions, Apache, profil réseau, relais WG	Ch. 15
Fig. 32	Serveur-startup 3/3 — Internet optionnel (tunnel, sync bootstrap), HTTPS 443 et résumé final	Ch. 15
Fig. 33	Application des changements FreePBX — Apply Config (fwconsole reload)	Ch. 15

Les informations absentes des dépôts sont signalées comme telles ; aucun secret (.env, tokens, mots de passe SIP/DB) n'est reproduit. © Projet ASAPHONE, juillet 2026.